HOOX · PYNE · DOCUMENTATION

SCRIPT AUTHORS · LIBRARY CONSUMERS · EDITOR USERS

END USER MANUAL

Install, CLI, library API, editors, and Pro API as a consumer

Open-source Pine Script evaluation framework

2026-10-17 DIN A4 · HTTPS://HOOX.SH/PYNE/DOCS OPEN SOURCE

CONTENTS

01 Overview

02 Overview

03 Installation

04 Quick Start

05 Configuration

06 Cli

07 Library Api

08 Evaluate Scripts

09 Editors

10 Pro Api Usage

11 Troubleshooting

12 Cli Commands

13 Modes

14 Glossary

15 Faq

OVERVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

---

title: "PYNE Documentation" description: "Open-source Pine Script™ evaluation — grammar, AST, interpret/compile runtime, alerts + webhooks, LSP, Pro API. pip install hoox-pyne · github.com/hoox-sh/pyne"

PYNE DOCUMENTATION

PYNE is the open evaluation stack for TradingView® Pine Script™: a formal ANTLR4 grammar, an ASDL algebraic AST, a deterministic bar-loop evaluator (interpret / compile / `mode=auto`), a Language Server Protocol surface, a Flask Pro API, and edge workers that share one evaluate contract — including **alerts** and optional **L2 webhooks**.

For **natural-language authoring** (PYNE Agent), see the sister project **pyne-agent-worker** — standalone Cloudflare® Workers AI™ agent with optional AXIS plugin; does not require pyne-worker.

Pine Script™ and TradingView® are trademarks of TradingView, Inc. This project is independent and not affiliated with or endorsed by TradingView, Inc. Cloudflare® is a trademark of Cloudflare, Inc.

Abstract

Where closed hosts couple the language to a proprietary charting host, PYNE treats the language as an inspectable pipeline:

```
Source (.pyne / .pine)
  → ANTLR4 lexer/parser (resource grammar)
  → ASDL AST (builder)
  → bar-loop (interpret | compile | auto)
  → plots / fill / events / drawings / alerts
  → optional L2 webhooks (Pro API + edge)
```

Live on PyPI as **`hoox-pyne`** (`import pynescript`). Source: github.com/hoox-sh/pyne.

```
pip install "hoox-pyne[lsp]"
pyne --version
# aliases: pynescript / pynescript-lsp
```

The same pipeline powers the desk CLI, the LSP binary, the Pro API, browser Pyodide (via AXIS), and Cloudflare® workers. Round-trip fidelity (`parse → unparse`) and interpret↔compile plot parity are first-class invariants — see compatibility. Local open-corpus set01–04 (2026-08-09): **parse 99.96%**, **Runtime interpret 100% excl. intentional demos** (set01 **249/249**). The repository does not ship third-party script corpora.

TypeScript consumers import **PyneTS** (`@hoox-sh/pynets`) — same public names; Python remains the oracle.

What's new

`hoox-pyne` 0.6.9 (2026-10-13) — three additions aimed at live and large workloads:

- **Response windowing** on `POST /run`, `/run/batch`, and every session route: optional `from_bar` / `to_bar` trim the **serialized envelope only**, leaving the bar loop and all Pine semantics untouched. On 20k bars a 500-bar window is **1,068.8 KB → 28.0 KB (38× smaller)**.
- **Live evaluation sessions** over HTTP — `POST /session`, `POST /session/<id>/tick`, `POST /session/<id>/append`. Re-ticking the forming bar is **O(1)**: 1,087.8 ms → **0.130 ms (8,343×)**; appending closed bars is **O(delta)** at **0.127 ms**.
- **Engine-owned OHLCV store** — `/data/series*` persists bars in SQLite keyed by `exchange|market|SYMBOL|interval`, and `/data/session` binds a live session to a stored series so bars stop travelling in the request body. These routes require an API key.

Post-0.6.9 on `main` — operational surface:

- **`GET /stats`** — public process-wide counters: requests, runs by mode, compile cache hit/miss, prewarm, delta sizes, sessions, datafeed. In-process, so treat it as a snapshot rather than a time series. See stats.
- **`PYNE_DEBUG=1`** — development-only logging that traces the interpreter bar loop, the compile pipeline, prewarm, delta calculation, and the datafeed. Optional `PYNE_DEBUG_SCOPE` narrows it to a logger allowlist. See observability.

0.6.7 (2026-09-23) — faster interpret dispatch (reused TA keys, steady `ta.sma` / `bb` / `atr` path) and compile `ta.uo` via `numba_uo`.

0.6.6 (2026-09-22) — last-sample TA (`ao` / `dmi` / `bb` / `cog` / `mode`), convert-to-v6 keeps UDF/`for n` names, webhook SSRF, LSP nested `strategy.risk` completion.

0.6.5 — convert-to-v6 leaves UDF definitions, parameters, and unpack names alone (set01–04 older scripts parse 323/323 after convert).

0.6.4 — convert any older Pine version to v6 from the editor, LSP code action, `pyne convert --to 6`, and `POST /lsp/convert`.

0.6.3 — optional Flask/VPS/container hosted runner (`PYNE_RUNNER`) plus CLI `pyne runner` (`deploy` / `tick` / `list`). Scripts keep evaluating after upload without AXIS holding the connection.

0.6.2 — interpret wraps subscripted UDF parameters (`s[1]`) per call site so `src = close; updown(src)` matches compile. Official TradingView® builtin scripts are **0 MISMATCH** vs compile.

0.6.1 — persistent `InterpretSession` (`Runtime.create_session`, delta `append_bars`, forming-bar `update_last_bar`). Dual-host `strategy.cash` qty-type, fill-time risk gates, session confirm-on-append, finalized-bucket HTF `lookahead_on`.

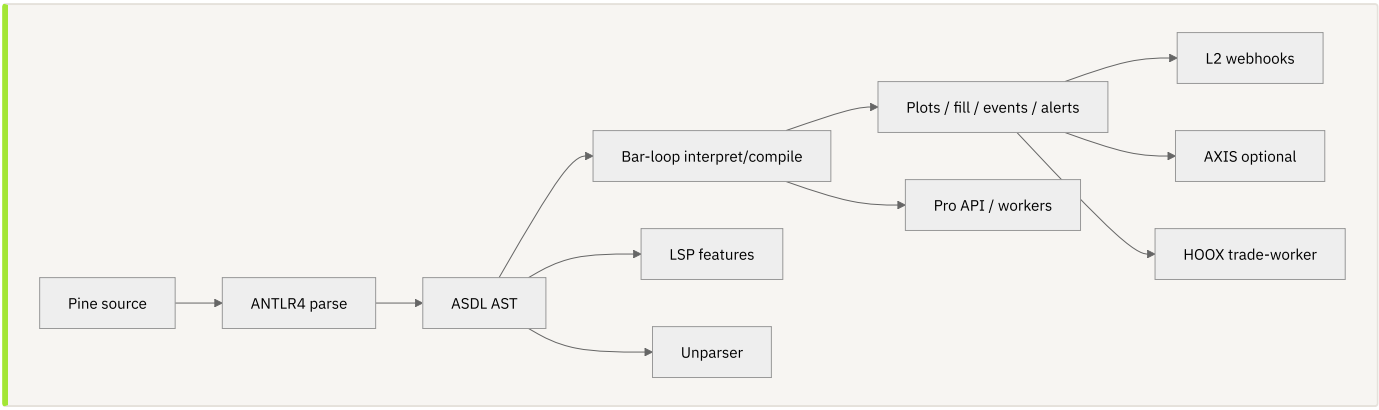
0.6.0 — type-aware intraday risk on both brokers, intrabar `var` rollback on realtime ticks, exchange-calendar `timeframe.change` D/W/M, `request.security` `gaps_on` / `lookahead_on` on HTF resample paths.

Full notes: GitHub releases · compatibility · evaluate scripts.

How to use

Surface	Role	Start here
Library + CLI	Parse, lint, unparse, evaluate. Library `Runtime.run` defaults to **interpret**; `pyne run` is **compile-only** smoke	End User hub · modes
Language core	Grammar, AST, visitors, type system	Core
Runtime	Series caps, builtins, strategy, alerts, compiler parity	Runtime
LSP	Diagnostics, completion, hover, format (`.pyne` / `.pine`)	LSP
Pro API	HTTP `/run` (alerts + L2 webhooks), `/optimize`, preview, backtest	API
Ops	CI, Docker, Nuitka, secrets, GCP	DevOps
PyneTS	TypeScript / Bun library (`parse` / `unparse` / `Runtime.run`)	PyneTS
AXIS (separate product)	Charting PWA AXIS	AXIS docs
HOOX	Edge trade mesh	HOOX docs

Conceptual model



Invariant: evaluation never requires a proprietary chart host. AXIS is an optional chart; HOOX is an optional *execution mesh*.

Tracks

End User

Install the package (`pip install hoox-pyne`), run the CLI, embed the library API, wire an editor (`.pyne` / `.pine`), or call the Pro API as a consumer — including alerts export and webhooks.

→ End User hub

Core / Runtime / LSP / API

Contributor and systems manuals: how the front-end, semantics, editor protocol, and HTTP bridge are built. Runtime covers series caps, incremental TA, drawing GC, `fill()` export, `request.security` HTF resample + barmerge (foreign/complex → `na`), strategy optimize, and parity.

DevOps

CI matrices, Fernet metadata encryption, Nuitka LSP binaries, containers, Cloud Build.

→ DevOps hub

Reference

Compatibility guarantees, missing surface, numerical validation, PyneTS, ecosystem map, roadmap.

Sister surfaces

- **PyneTS** — TypeScript / Bun library. Standalone hoox-sh/pynets / `@hoox-sh/pynets` . See PyneTS.
- **pyne-worker** (sister repo) — Python Cloudflare® Worker for production `POST /run` (alerts + cron + R2).
- **pyne-agent-worker** (sister repo) — NL authoring host (`POST /v1/chat`); optional validate via pyne-worker.
- **AXIS** — installable PWA; documented at hoox.sh/axis/docs.
- **Family map** — Ecosystem · modes.

Offline & agent exports

Auto-generated like HOOX manuals (`bun run docs:exports`):

Kind	Path
Full-corpus LLM pack	`llm.txt`
LLM site map (llmstxt.org)	`llms.txt`
Track PDFs (A4)	`/exports/pyne-*-manual.pdf`

See also

- Installation
- Quick start
- Runtime modes
- Alerts & webhooks
- Grammar pipeline
- Evaluate contract
- POST /optimize
- pyne-worker
- pyne-agent-worker
- Roadmap

OVERVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "End User Hub" description: "Install PYNE (pip install hoox-pyne). Library default is interpret, pyne run is compile-only, Pro API /run defaults to auto."

END USER HUB

This track is for people who **consume** PYNE: parse and reformat Pine Script™, lint before upload, evaluate expressions or full scripts against OHLCV (library default **interpret**, `pyne run` is compile-only, Pro API `/run` defaults to **auto**), export **alerts** (optional L2 webhooks), wire an editor via LSP (`.pyne` / `.pine`), or call the HTTP evaluate contract. Contributor internals (grammar regeneration, AST builder, bar-loop semantics) live under Core, Runtime, and DevOps.

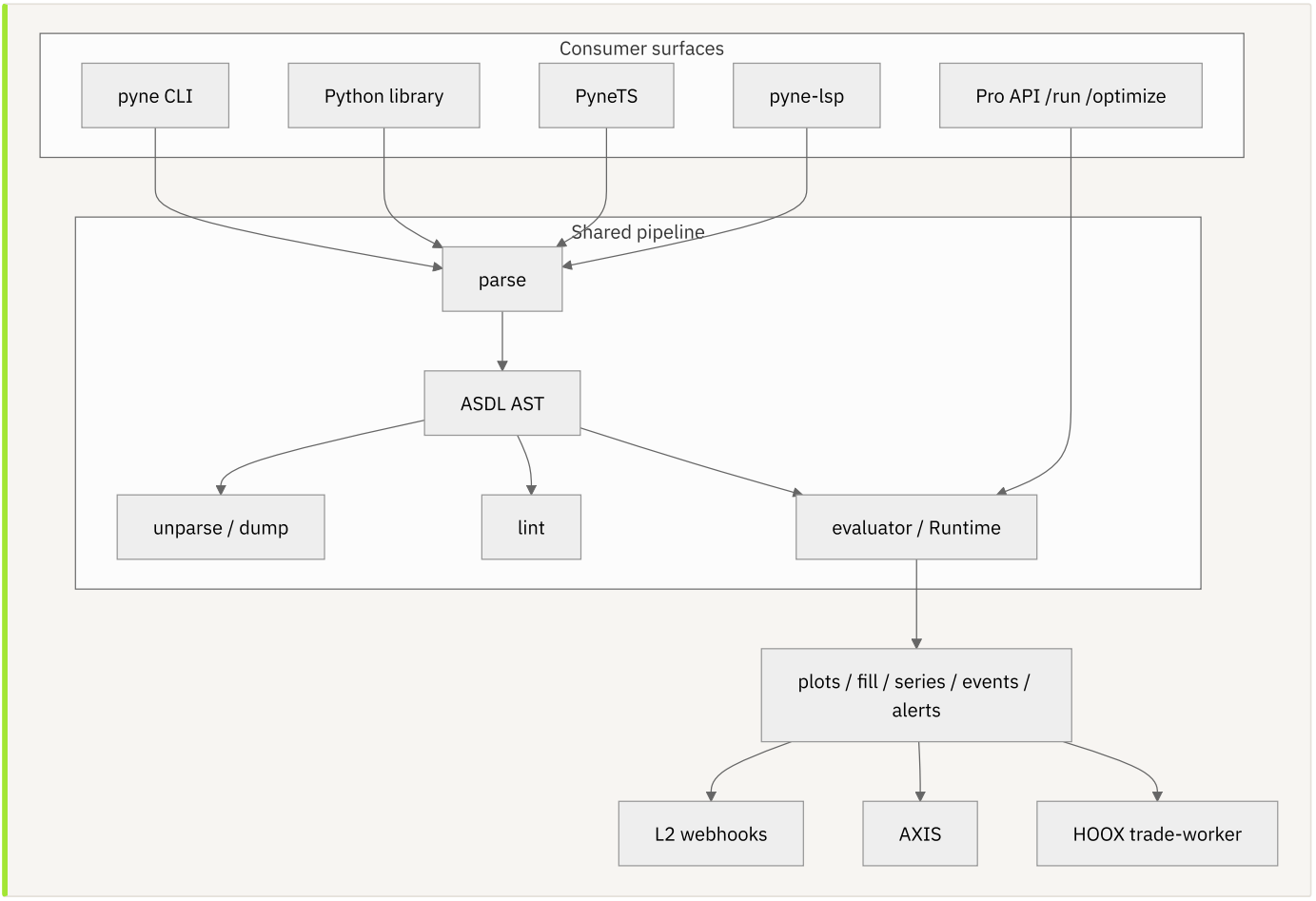
Abstract

PYNE exposes one language pipeline through several thin surfaces:

```
.pyne / .pine source
  → parse (ANTLR4 + ASDL AST)
  → optional: dump | unparse | lint | walk/transform
  → optional: evaluate (literal_eval | Runtime default interpret | Pro API mode=auto)
  → plots / fill / events / drawings / alerts / metrics
  → optional L2 webhooks (Pro API / edge)
```

The **desk path** is the `pyne` Click CLI (alias `pynescript`) and `pynescript.ast` / `pynescript.runtime` library API (PyPI distribution name `hoox-pyne`). The **TypeScript desk path** is **PyneTS** (`bun add @hoox-sh/pynets`). The **editor path** is `pyne-lsp` (alias `pynescript-lsp`; stdio; first-class `.pyne` association). The **HTTP path** is the Flask Pro API (`POST /run` with alerts export + webhooks, `POST /optimize`, preview, backtest). The optional chart is AXIS; the optional trade mesh is HOOX. Evaluation does not require either. Defaults for `mode` differ by surface — modes.

Conceptual model



Surface	Entry	Typical job
CLI	`pyne …` (alias `pynescript`)	One-shot parse, format, lint, compile, `run`, `optimize`, prewarm, data
Library	`from pynescript.ast import parse, unparse`	Embed parse/transform/eval in tools
PyneTS	`import { parse, Runtime } from "@hoox-sh/pynets"`	Same names in TypeScript / Bun
LSP	`pyne-lsp` (alias `pynescript-lsp`)	Diagnostics, completion, hover (`.pyne` / `.pine`)
Pro API	`POST /run` · `POST /optimize`	Evaluate contract + strategy `input.*` search
AXIS	separate product	Chart PWA AXIS over evaluate results
HOOX	separate product	Edge execution mesh after strategy events

Choose your path

1. Getting started

New install, first parse, and environment knobs:

- **Installation** — PyPI **`pip install hoox-pyne`** (import `pynescript`), extras (`lsp`, `compile`, `data`, `pro`), Hatch checkout, smoke checks.
- **Quick start** — Parse → dump → unparse → lint → `literal_eval` in under ten minutes.
- **Configuration** — Extras, console scripts, Pro API env vars (`ALERT_WEBHOOK_URL`, compile cache), editor settings.

2. Operational guides

Daily workflows against the same pipeline:

- **CLI** — `check`, `format`, `lint`, `compile`, `prewarm`, `run`, `optimize`, `data`, …
- **Library API** — `parse`, `unparse`, `dump`, `walk`, visitors, transformers, linter, `optimize.run_study`.
- **Evaluate scripts** — Expression eval, bar-loop Runtime (library default **interpret**), strategy events, **alerts export**, mock vs live data.
- **Editors** — VS Code (`.pyne` first-class), Neovim, Zed, Emacs, Helix; `clients/` snippets.
- **Pro API usage** — `/run` (+ `webhook_url` / L2 webhooks), `/run/batch`, `/optimize`, preview, backtest as a **consumer**.
- **Troubleshooting** — Parse failures, recursion limits, missing extras, CORS, auth.

3. Reference

- **CLI commands** — Full option trees for every Click command.
- **Modes** — `interpret` / `compile` / `auto` defaults: library **interpret**, `POST /run` **auto**, `pyne run` **compile-only**.
- **Glossary** — AST, series, bar-loop, Runtime, ASDL, and related terms.
- **FAQ** — Version support, TradingView parity, licensing, AXIS/HOOX boundaries.

Interface surface (consumer map)

Want	Command / import	Docs
Install core	`pip install hoox-pyne` (import `pynescript`)	Installation
Install LSP	`pip install "hoox-pyne[lsp]"`	Editors
Install market data deps	`pip install "hoox-pyne[data]"`	CLI data
Parse file	`pyne parse-and-dump path.pyne`	CLI
Normalize source	`pyne parse-and-unparse path.pine`	CLI
Lint	`pyne lint path.pine`	CLI
Library parse	`from pynescript.ast import parse, unparse`	Library API
Expression eval	`from pynescript.ast.helper import literal_eval`	Evaluate
Bar-loop eval	`from pynescript.runtime import Runtime` (`mode` default **interpret**)	Evaluate · modes
Incremental interpret	`Runtime.create_session` → `append_bars` / `update_last_bar` (0.6.1)	Evaluate
Compile-only smoke	`pyne run script.pine --bars 50`	CLI
Warm Numba / IR	`pyne prewarm` / `POST /compile/prewarm`	CLI
Strategy HPO	`pyne optimize` / `POST /optimize` / `pynescript.optimize.run_study`	Optimize
Alerts + webhooks	`/run` → `alerts[]` / `webhook_url`	Alerts
Start LSP	`pyne-lsp` (stdio)	Editors
Local Pro API	`make run` / `python -m backend.app`	Pro API usage

Preferred console scripts (aliases in parentheses) are registered in `pyproject.toml`:

Script	Module	Role
`pyne` (`pynescript`)	`pynescript.__main__:cli`	Desk CLI (Click group)
`pyne-lsp` (`pynescript-lsp`)	`pynescript.langserver.__main__:main`	Language server (pygls)

Do not conflate them: lint and parse live on `pyne`; editor features live on `pyne-lsp`.

Internals (repo paths)

Consumer-relevant code only — deeper design is linked from each guide:

Path	Role
`src/pynescript/__main__.py`	Click CLI: check, format, lint, compile, prewarm, run, optimize, data, info
`src/pynescript/ast/helper.py`	Public `parse`, `unparse`, `dump`, `walk`, `literal_eval`
`src/pynescript/ast/linter.py`	`lint_script` / `PineLinter`
`src/pynescript/ast/evaluator/`	Bar-aware and literal evaluators
`src/pynescript/runtime/`	Package Runtime SoT (bar-loop host, series, CustomEvaluator)
`src/pynescript/langserver/`	LSP features and server
`src/pynescript/util/data.py`	Historical data providers (mock, yahoo, alphavantage, ccxt)
`backend/app.py`	Flask Pro API entry (`/run`, auth, blueprints)
`backend/runtime.py`	Compat shim → `pynescript.runtime`
`examples/`	Worked scripts (parse, RSI strategy, datafeed)
`clients/`	Editor client configs (Neovim, Zed, Emacs)
`vscode-extension/`	VS Code extension package

Invariants & edge cases

- **Round-trip is first-class.** `parse` → `unparse` should preserve semantics; use it to normalize formatting, not as a semantic rewrite.
- **Mode split.** `parse(..., mode="exec")` yields a script tree; `mode="eval"` parses a single expression (used by `literal_eval`).
- **No proprietary host required.** Evaluation works offline with mock or supplied OHLCV; AXIS/HOOX are optional.
- **Extras are opt-in.** Core parse/lint needs only base deps (`antlr4-python3-runtime`, `click`, …). LSP needs `[lsp]`; CCXT paths need `[data]` / `[datafeed]`.
- **Optional example dir.** `pinescript_filepath` only expands when `--example-scripts-dir` points at local `*.pine` files; no third-party corpus is shipped.

Worked examples

Minimal library round-trip:

```
from pynescript.ast import parse, unparse

source = """
//@version=6
indicator("My RSI")
plot(ta.rsi(close, 14))
"""
tree = parse(source)
print(unparse(tree))
```

Minimal CLI:

```
pip install hoox-pyne
pyne parse-and-dump examples/rsi_strategy.pine
pyne lint examples/rsi_strategy.pine
```

Minimal HTTP evaluate (local server running):

```
curl -s http://127.0.0.1:5002/ \
  | python -m json.tool
```

Failure modes

Symptom	Likely cause	Where to go
`pyne: command not found`	Package not on PATH / venv inactive	Installation
`pyne-lsp` missing	Installed without `[lsp]` extra	Editors
`SyntaxError` on parse	Grammar mismatch or truncated source	Troubleshooting
`/run` 400 `NO_DATA`	Missing OHLCV list	Pro API usage
Numerical drift vs TradingView	Builtin / series edge case	Compatibility

See also

- PYNE product map — full stack overview
- Installation
- Quick start
- Runtime modes
- Evaluate contract (API)
- AXIS docs — optional chart
- HOOX docs — optional edge trade mesh

INSTALLATION

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Installation" description: "Install hoox-pyne from PyPI (live) or a source checkout — CLI, LSP, compile, data, and Pro API extras."

INSTALLATION

Abstract

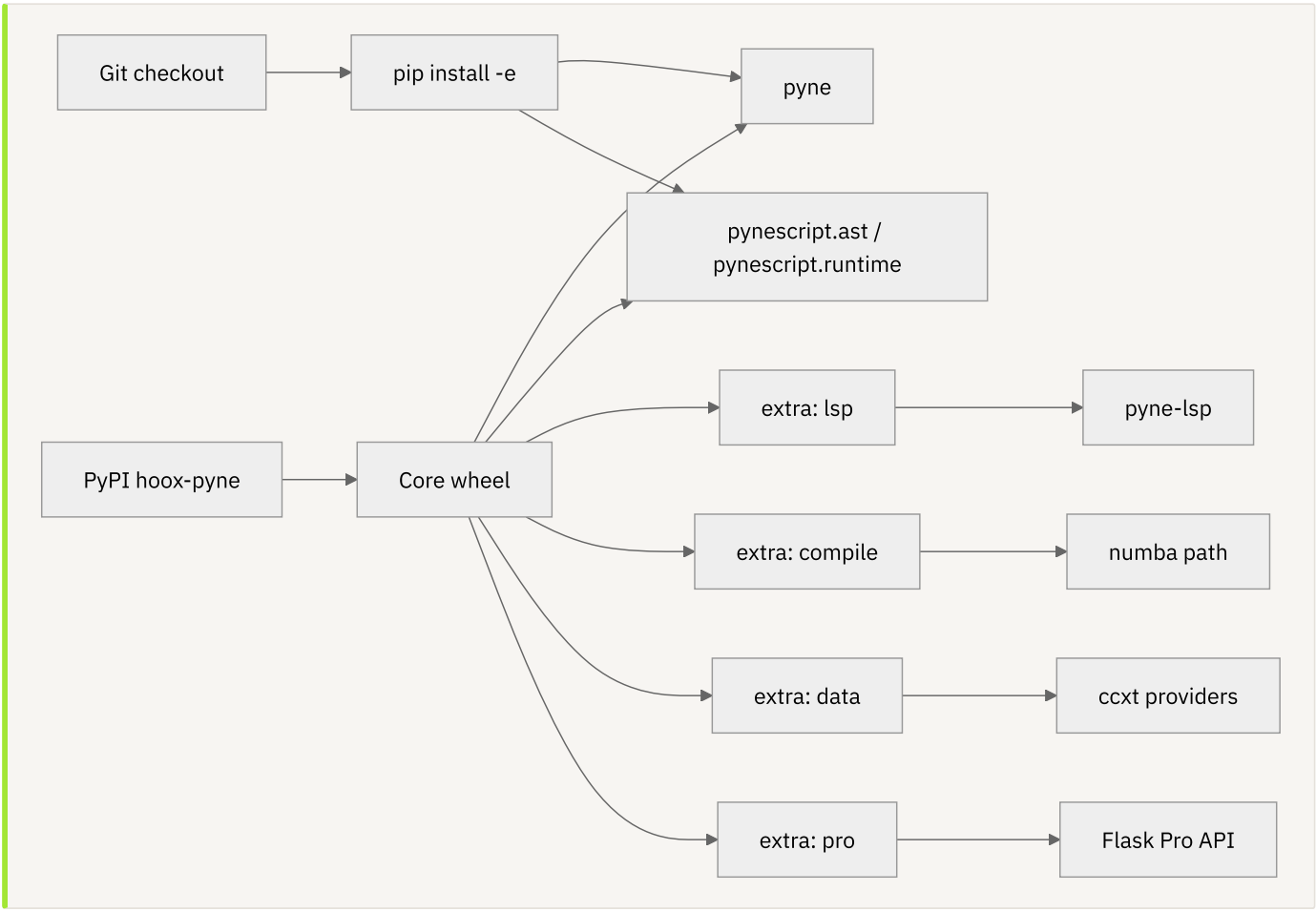
PYNE is live on PyPI as `hoox-pyne` (currently **0.6.7**). The **import package** remains `pynescript` (src-layout under `src/pynescript/`). Preferred console scripts are `pyne` / `pyne-lsp`; legacy `pynescript` / `pynescript-lsp` remain installed as aliases.

Do **not** install:

- plain `pyne` / `PyNE` from PyPI without the `hoox-` prefix — unrelated 0.1.0 process-networking library (our wheel is `hoox-pyne`)
- upstream `pynescript` (elbakramer) — different project

```
pip install hoox-pyne
pip install "hoox-pyne[lsp]"       # language server
pip install "hoox-pyne[compile]"   # Numba compile path
pip install "hoox-pyne[data]"      # ccxt market data
pip install "hoox-pyne[pro]"       # Flask Pro API stack
```

Conceptual model



Install flavor	What you get	When
`pip install hoox-pyne`	CLI + library (`pynescript.runtime` SoT)	Desk use, scripts, CI lint
`pip install "hoox-pyne[lsp]"`	+ pygls LSP server	Editors
`pip install "hoox-pyne[compile]"`	+ numpy / numba	`compile` / `prewarm` / `run` Numba modes
`pip install "hoox-pyne[data]"`	+ ccxt	Exchange data providers
`pip install "hoox-pyne[pro]"`	+ Flask Pro API stack	Self-hosted `/run`
`pip install -e ".[dev-lsp]"`	Editable + LSP test harness	Contributing to LSP
Docker CLI image	Full CLI image	CI gates / isolated runs
Hatch env	Matrix tests, lint, docs	Full monorepo workflow

Interface surface

Prerequisites

- **Python ≥ 3.10** (classifiers declare 3.10–3.13)
- `pip` or an equivalent installer; virtualenv strongly recommended
- Optional: Docker for the CLI image; Node 22+ only if building the VS Code extension

Core install (PyPI)

```
python3 -m venv .venv
source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -U pip
pip install "hoox-pyne[lsp,compile]"
pyne --version
pyne info
```

Project page: pypi.org/project/hoox-pyne · owner account jango-blockchained.

Extras

Defined in `pyproject.toml` under `[project.optional-dependencies]`:

```
pip install "hoox-pyne[lsp]"
pip install "hoox-pyne[compile]"
pip install "hoox-pyne[data]"      # alias: datafeed
pip install "hoox-pyne[pro]"
pip install "hoox-pyne[lsp,compile,data]"
```

Source checkout

```
git clone https://github.com/hoox-sh/pyne.git
cd pyne
pip install -e ".[lsp,pro]"
# or:
make install
```

Docker CLI

```
docker pull   # when published to a registry; or build from source:
git clone https://github.com/hoox-sh/pyne.git && cd pyne
make docker-build-cli
docker run --rm -v "$PWD:/work" -w /work pynescript-cli:latest check script.pine
# ≡ make docker-cli ARGS="check script.pine"
```

Console scripts

Command	Purpose
`pyne`	Preferred Click CLI (check, format, lint, compile, prewarm, run, data, info)
`pyne-lsp`	Preferred language server (requires `[lsp]`)
`pynescript`	Alias of `pyne` (same callable)
`pynescript-lsp`	Alias of `pyne-lsp` (same callable)

```
pyne --version
pyne --help
pyne check script.pine
which pyne-lsp   # only after [lsp]
```

Standalone binaries and a CLI image tarball also ship on GitHub Releases.

Internals (repo paths)

Path	Role
`pyproject.toml`	Package metadata (`name = "hoox-pyne"`), extras, scripts
`src/pynescript/`	Library + CLI + LSP package tree
`src/pynescript/__about__.py`	`__version__`
`src/pynescript/__main__.py`	CLI implementation
`src/pynescript/runtime/`	Package Runtime SoT (bar-loop host, series, evaluator)
`src/pynescript/langserver/__main__.py`	`pyne-lsp` entry
`Dockerfile` target `cli`	Ephemeral CLI image
`Makefile`	`package`, `docker-build-cli`, `install`, …
`CONTRIBUTING.md`	Publish / Hatch workflow

Invariants & edge cases

- **PyPI name ≠ import name.** Install **`hoox-pyne`**; `import pynescript`. CLIs: **`pyne`** / **`pyne-lsp`**.
- **License.** AGPL-3.0-or-later. Network use triggers AGPL source-offer obligations (see GNU AGPL §13).
- **LSP is not a CLI subcommand.** Use **`pyne-lsp`** (or `python -m pynescript.langserver`).
- **Pro API.** `pip install "hoox-pyne[pro]"` or monorepo `pip install -r backend/requirements.txt` + `make run` (default `:5002`).
- **Source extensions.** Prefer **`.pyne`** for stack sources; `.pine` / `.pinev5` / `.pinev6` still work.

Worked examples

Smoke-test CLI and library

```
python - <<'PY'
from pynescript.ast import parse, unparse
from pynescript import __about__
print("version", __about__.__version__)
src = '//@version=6\nindicator("t")\nplot(close)\n'
print(unparse(parse(src)))
PY

pyne check examples/rsi_strategy.pine
pyne lint examples/rsi_strategy.pine
# optional (needs [compile] for Numba):
# pyne prewarm
# pyne run examples/rsi_strategy.pine --bars 50
```

Monorepo desk + Pro API

```
git clone https://github.com/hoox-sh/pyne.git
cd pyne
pip install -e ".[lsp,pro]"
make run   # Flask on :5002
```

Failure modes

Failure	Diagnosis	Fix
`ModuleNotFoundError: pynescript`	Wrong env / not installed	Activate venv; `pip install hoox-pyne`
`No module named 'pygls'`	LSP without extra	`pip install "hoox-pyne[lsp]"`
`No module named 'numba'`	compile path without extra	`pip install "hoox-pyne[compile]"`
`No module named 'ccxt'`	data path without extra	`pip install "hoox-pyne[data]"`
Wrong package / missing APIs	Installed upstream `pynescript` or plain `pyne`	`pip uninstall pynescript pyne` then `pip install hoox-pyne`
`pyne` / `pynescript` not found	Scripts not on PATH	`python -m pynescript` or fix venv `bin/`

See also

- Quick start
- CLI guide
- Editors / LSP
- PyPI publish (maintainers)
- End User hub

QUICK START

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

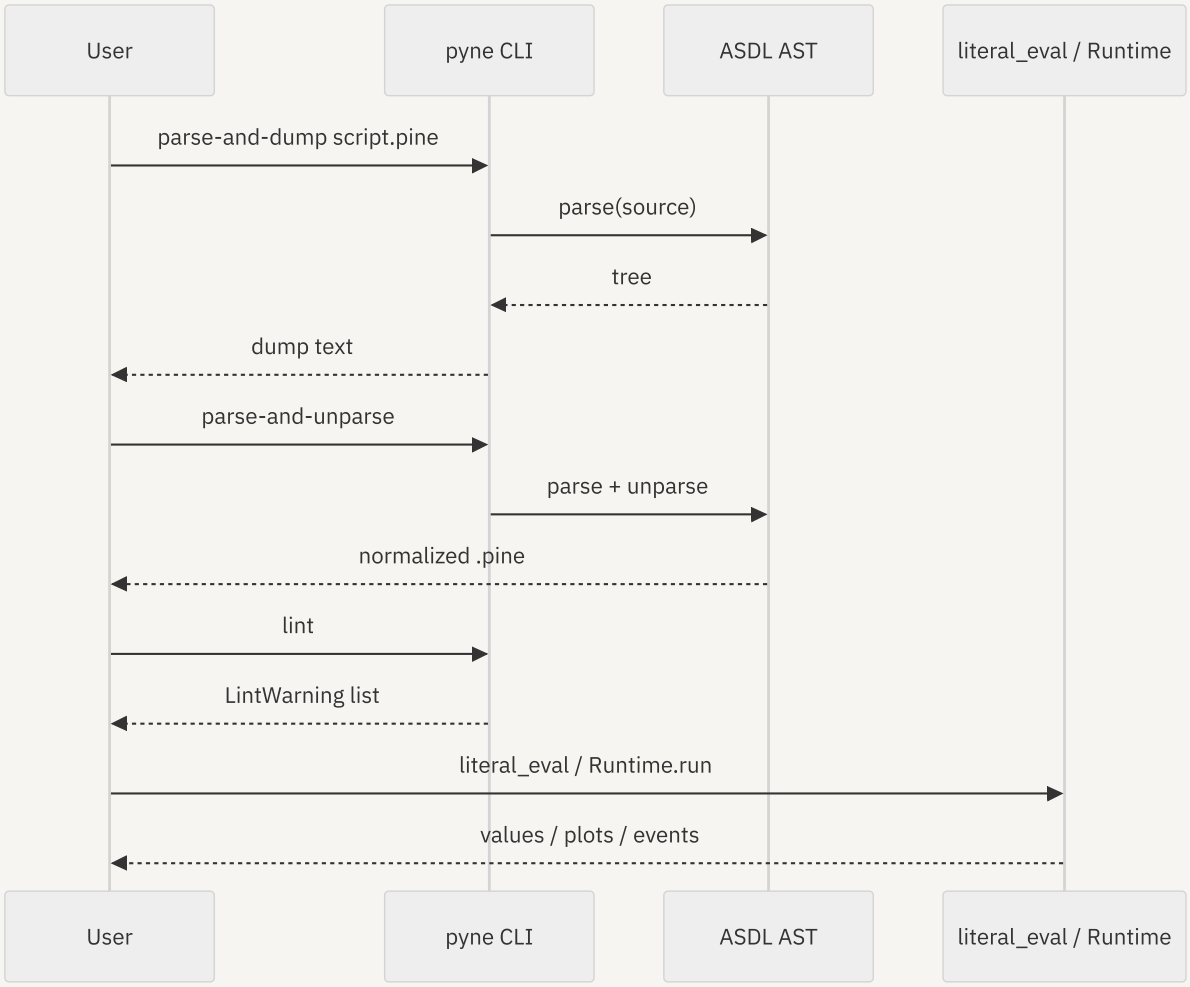
title: "Quick Start" description: "Parse, dump, unparse, lint, and evaluate Pine Script™ with the pyne CLI and library in under ten minutes."

QUICK START

Abstract

This page is a linear path from a working install to the five desk operations most users need: **parse**, **dump**, **unparse**, **lint**, and **evaluate**. Full option trees live in CLI commands; deeper evaluation semantics live in Evaluate scripts.

Conceptual model



Interface surface

Assumes:

```
pip install hoox-pyne                # https://pypi.org/project/hoox-pyne/ · 0.6.7
# optional:
# pip install "hoox-pyne[lsp]"       # editors
# pip install "hoox-pyne[compile]"   # compile / run / prewarm
# pip install "hoox-pyne[data]"      # ccxt providers
```

```
pyne info
pyne check script.pine
pyne lint script.pine
# aliases still work: pynescript info …
```

Sample script in-tree: `examples/rsi_strategy.pine`. Prefer **`//@version=6`** and **`.pyne`** / **`.pine`** for new work (`.pinev5` remains a supported legacy association).

Internals (repo paths)

Step	Implementation
CLI parse/dump	`src/pynescript/__main__.py` → `parse_and_dump`
CLI unparse	`parse_and_unparse`
CLI lint	`lint` → `pynescript.ast.linter.lint_script`
Library helpers	`src/pynescript/ast/helper.py`
Bar-loop Runtime SoT	`src/pynescript/runtime/` (`from pynescript.runtime import Runtime`)
Example scripts	`examples/parse_dump_unparse.py`, `examples/evaluate_expressions.py`

Invariants & edge cases

- Always declare `//@version=6` at the top (v5 still parses); the linter warns (`W001`) if the pragma is missing.
- `parse-and-unparse` is a **formatter via AST**, not a semantic optimizer.
- `literal_eval` is for expressions / built-in calls with optional series context — not a full multi-bar strategy backtester. For bar loops, use `pynescript.runtime.Runtime` (or the Pro API `/run`).
- Filename arguments to the CLI must exist and be readable; lint may also read stdin (`-` or no file).

Worked examples

1. Parse and dump the AST

```
pyne parse-and-dump examples/rsi_strategy.pine
```

Pretty-print with indent and optional file output:

```
pyne parse-and-dump examples/rsi_strategy.pine --indent 2 --output-file /tmp/rsi.ast.txt
```

Library equivalent:

```
from pynescript.ast import parse, dump

with open("examples/rsi_strategy.pine", encoding="utf-8") as f:
    tree = parse(f.read(), "examples/rsi_strategy.pine")
print(dump(tree, indent=2))
```

2. Round-trip unparse (normalize)

```
pyne parse-and-unparse examples/rsi_strategy.pine
pyne parse-and-unparse messy.pine --output-file clean.pine
```

```
from pynescript.ast import parse, unparse

source = """
//@version=6
indicator("My RSI")
rsi(close, 14)
"""
print(unparse(parse(source)))
```

Canonical demo: `examples/parse_dump_unparse.py`.

3. Lint before upload

```
pyne lint examples/rsi_strategy.pine
pyne lint --fail-on warnings examples/rsi_strategy.pine
echo '//@version=6\nindicator("x")\nplot(close)' | pyne lint -
```

Library:

```
from pynescript.ast.linter import lint_script

issues = lint_script(open("examples/rsi_strategy.pine").read(), "rsi_strategy.pine")
for w in issues:
    print(w)  # severity: [CODE] message at line N
```

4. Evaluate expressions

```
from pynescript.ast.helper import literal_eval

print(literal_eval("1 + 2 * 3"))  # 7
print(literal_eval("math.sqrt(16)"))
prices = [100, 102, 101, 103, 105, 104, 106, 108, 107, 110]
print(literal_eval(f"ta.rsi({prices}, 9)"))

# Series history with context
ctx = {
    "close": [100, 102, 101, 103, 105],
    "open": [99, 101, 100, 102, 104],
}
print(literal_eval("close[0] - close[1]", ctx))
```

Broader demo: `examples/evaluate_expressions.py`.

5. Fetch sample market data (CLI)

```
# mock provider (no network)
pyne data AAPL --provider mock

# Yahoo (network)
pyne data AAPL --provider yahoo --period 6mo --interval 1d

# CCXT (needs hoox-pyne[data])
pyne data BTC/USDT --provider ccxt --exchange binance
```

6. Optional: local Pro API `/run`

```
# from monorepo with backend deps
make run
```

```
curl -s -X POST http://127.0.0.1:5002/run \
  -H 'Content-Type: application/json' \
  -d '{
    "script": "//@version=6\nindicator(\"t\")\nplot(close)",
    "mode": "auto",
    "data": [
      {"time": 1, "open": 1, "high": 2, "low": 0.5, "close": 1.5, "volume": 100},
      {"time": 2, "open": 1.5, "high": 2.5, "low": 1.0, "close": 2.0, "volume": 120}
    ]
  }' | python -m json.tool
```

Omit `mode` to use the schema default **`auto`** (warm compile when eligible, else interpret). Response may include `series`, `plot_meta`, `events`, `drawings`, and `alerts`. See Pro API usage for batch mode, webhooks, free-tier guards, and prewarm.

Prefer saving stack sources as **`.pyne`** (`.pine` still works).

7. Library bar-loop vs CLI smoke

Library `Runtime.run` defaults to **interpret**. `pyne run` is **compile-only** on synthetic bars and does not accept `--mode`.

```
from pynescript.runtime import Runtime

ohlcv = [
    {"time": 1, "open": 1, "high": 2, "low": 0.5, "close": 1.5, "volume": 100},
    {"time": 2, "open": 1.5, "high": 2.5, "low": 1.0, "close": 2.0, "volume": 120},
]
out = Runtime(symbol="DEMO").run(
    '//@version=6\nindicator("t")\nplot(close)\n',
    ohlcv,
    mode="interpret",
    timeout_seconds=5.0,
)
print(out.get("mode"), list(out.get("series", {})))
```

```
pip install "hoox-pyne[compile]"
pyne run examples/rsi_strategy.pine --bars 50
```

See modes.

8. Optional: start LSP for editors

```
pip install "hoox-pyne[lsp]"
pyne-lsp   # stdio; normally launched by the editor
# alias: pynescript-lsp
```

Wire Neovim / Zed / Emacs using Editors and `clients/`.

Failure modes

Step fails	Check
Dump raises `SyntaxError`	Script version / unsupported construct — try a minimal `indicator` + `plot(close)`
Unparse output differs cosmetically	Expected; compare semantic structure via dump
Lint exits non-zero	`--fail-on` threshold; inspect codes `E001`, `W001`, …
`literal_eval` `NotImplementedError`	Expression not in literal/builtin subset; use full Runtime
`data` provider error	Network, API key, missing `ccxt` extra

See also

- Installation
- Configuration
- CLI guide
- Library API
- Evaluate scripts
- Runtime modes
- CLI command reference

CONFIGURATION

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

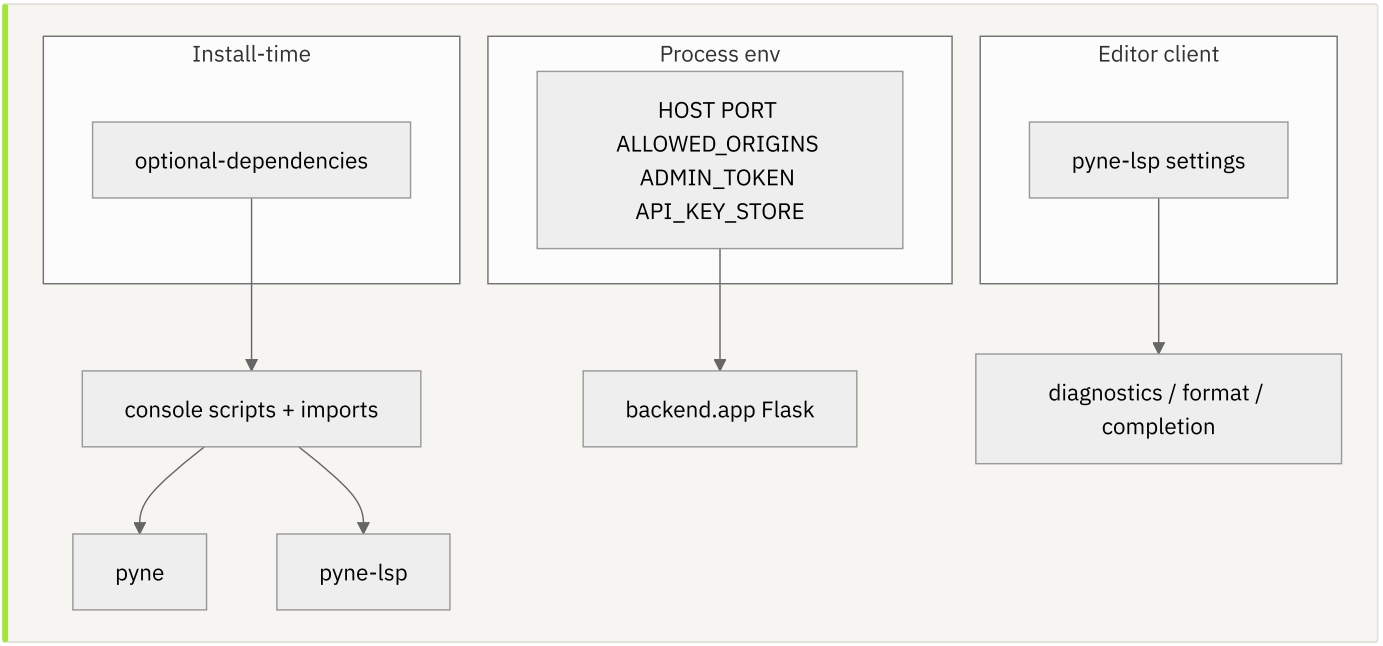
title: "Configuration" description: "Package extras, console scripts, Pro API environment variables, data providers, and editor LSP settings for PYNE."

CONFIGURATION

Abstract

PYNE is mostly **convention-over-config** at the library layer: parse and evaluate take explicit arguments rather than a global config file. Configuration appears at three boundaries: **install extras** (what code is available), **process environment** (Pro API host, CORS, API keys store), and **editor / LSP client settings** (diagnostics, formatting, binary path). This page inventories those knobs for end users.

Conceptual model



There is no `pynescript.toml` (or similar) for core use. Parse, evaluate, and CLI flags are per-invocation.

Interface surface

Package extras (`pyproject.toml`)

Extra	Dependencies	Enables
(none)	antlr4-runtime, click, requests, tqdm	CLI + AST + linter + literal_eval + `pynescript.runtime`
`lsp`	pygls, lsprotocol	`pyne-lsp` (alias `pynescript-lsp`)
`dev-lsp`	pygls, lsprotocol, pytest-lsp	LSP + protocol tests
`data`	ccxt	Exchange historical data
`datafeed`	ccxt	Same as `data` (alias for realtime feed paths)
`compile`	numpy, numba	Compile / run / prewarm Numba path
`pro`	Flask stack (+ redis optional)	Self-hosted Pro API

Console scripts

Name	Entry point	Notes
`pyne`	`pynescript.__main__:cli`	Preferred desk CLI
`pyne-lsp`	`pynescript.langserver.__main__:main`	Preferred language server
`pynescript`	same as `pyne`	Legacy alias
`pynescript-lsp`	same as `pyne-lsp`	Legacy alias

Invoke without scripts on PATH:

```
python -m pynescript --help
python -m pynescript.langserver
```

CLI flags that act as “config”

Per-invocation only (no persistence):

Command	Notable options
`check`	`--encoding`, `-q/--quiet`, `--ext` (directory walk)
`format` / `fmt`	`-w/--write`, `--check`, `-o/--output-file`
`parse-and-dump`	`--encoding`, `--indent`, `--output-file`
`parse-and-unparse`	`--encoding`, `--output-file`
`lint`	`--encoding`, `--fail-on {errors,warnings,all,never}`, `--json`, `-q`
`compile`	`--emit`, `-o`, `--time/--no-time`
`prewarm`	`--force`, `--json`
`run`	`--bars` (default 50), `--json`, `-q` — **compile-only** synthetic OHLCV
`data`	`--provider`, `--period`, `--interval`, `--api-key`, `--secret`, `--exchange`, `--format`

`lint` has **no** `--fix`. Structural rewrite is `pyne format -w` (parse → unparse).

Pro API environment variables

Consumed by `backend/app.py` and middleware:

Variable	Default	Purpose
`HOST`	`127.0.0.1`	Bind address when running `__main__`
`PORT`	`5002`	Bind port
`ALLOWED_ORIGINS`	`https://pynescript.ai`, `https://app.pynescript.ai`, plus localhost regex	CORS allowlist (comma-separated; regex allowed). Code also appends localhost, private-LAN, and product-origin (`hoox.sh` / `pynescript.ai` / `pynescript.online`) regexes unless `*`
`ADMIN_TOKEN`	unset	Required for `POST /auth/create_key`; if unset, create returns 403
`API_KEY_STORE`	`/root/pynescript/data/api_keys.json`	Path for key store (override for local)
`ALERT_WEBHOOK_URL`	unset	Default L2 alert webhook for `/run` when body omits `webhook_url`
`ALERT_WEBHOOK_TIMEOUT`	`10`	Webhook HTTP timeout (seconds)

Also relevant:

Variable	Context
`MAX_CONTENT_LENGTH`	Hardcoded 5 MiB on Flask app config (not env) — large OHLCV POSTs fail above this

Runtime / evaluate environment variables

Used by `pynescript.runtime.Runtime` (`src/pynescript/runtime/host.py`) and the compiler host. `backend.runtime` is a compat re-export only.

Variable	Default	Purpose
`PYNE_RUNTIME_MODE`	`interpret`	Default when `Runtime.run(..., mode=None)` omits mode (**Pro API schema default is still `auto`**; `pyne run` ignores this — compile-only)
`PYNE_SERIES_CAP`	**on**	Trim append-only host series lists; set `0` / `false` / `off` to disable (oracle / debug)
`PYNE_SERIES_MAX`	unset	Absolute series history depth (overrides default 256 + `max_bars_back`)
`PYNE_TA_INCREMENTAL`	**on**	Bar-mode incremental `ta.*` hot path; set `0` to force full recompute
`PYNE_SERIES_RING`	**off**	Chronological tail view; skip dual list write. Keep off unless you need the ring path
`PYNE_PARSE_CACHE`	**on**	Process-local LRU of successful `parse()` trees; set `0` to disable
`PYNE_PARSE_CACHE_MAX`	`128`	Max parse-cache entries
`PYNE_COMPILE_DISK_CACHE`	often `1` in deploy	Persist compiled IR across processes
`PYNE_COMPILE_CACHE_DIR`	XDG cache / Docker `/data/compile-cache`	Disk IR location
`PYNE_COMPILE_PREWARM`	deploy-dependent	Host cold-start prewarm of Numba builtins

See Series & history, Technical builtins, and Compiler overview.

Local run:

```
python -m backend.app
# or: make run
```

Data providers

CLI `pynescript data` / library providers:

Provider	Auth	Notes
`mock`	none	Deterministic offline bars (default)
`yahoo`	none	Yahoo Finance path
`alphavantage`	`--api-key` (falls back to `demo`)	Limited with demo key
`ccxt`	optional key/secret; `--exchange`	Requires `[data]` extra

Pro API `/run` optional fields:

Field	Values	Role
`data_source`	`""`, `mock`, `ccxt`, `ccxtpro`, `yahoo`, `alphavantage`	Wires `request.*` resolution
`data_options`	object	exchange, api_key, seed, …
`mode`	`auto` (default), `interpret`, `compile`	Prefer warm compile + fallback; strict compile; full interpret
`inputs`	object	`input.*` overrides by title (forces interpret under `auto`)
`libraries`	list	`[{namespace, name, version, source}]` — max 32; also on `/run/batch`; interpret / `auto` fallback
`timeout_seconds`	number	Optional interpret wall-clock budget; omit / `null` / `≤ 0` → no timeout
`profiler`	bool	Per-line timings; forces interpret
`webhook_url`	string	Per-request L2 alert webhook (overrides `ALERT_WEBHOOK_URL`)
`forward_alerts` / `alert_last_bar` / `alert_batch`	bool	Webhook delivery controls (see Alerts)

`timeout_seconds` is accepted on `Runtime.run`, `POST /run`, and `POST /run/batch`. Extra keys that are not in the schema still → `UNKNOWN_FIELDS`.

Operators can also `POST /compile/prewarm` or run `pyne prewarm` (and `pyne prewarm script.pine …`) to warm Numba builtins / script IR caches before interactive traffic. Requires `pip install "hoox-pyne[compile]"` for full Numba warm-up.

Free-tier Pro API guards (unauthenticated `/run`) are **off by default**. Set `FREE_TIER_LIMITS=1` to enable `FREE_MAX_BARS`, `FREE_MAX_SCRIPT_CHARS`, `FREE_MAX_CONCURRENT`, `FREE_RATE_LIMIT` / `FREE_RATE_WINDOW_SEC` — see Pro API usage.

Editor / LSP client settings

VS Code extension keys (from `clients/README.md`):

Setting	Default	Meaning
`pynescript.lsp.enabled`	`true`	Toggle server
`pynescript.lsp.command`	`auto`	`auto` tries `pyne-lsp` → `pynescript-lsp` → `python3 -m pynescript.langserver`; or an absolute path
`pynescript.lsp.python`	`python3`	Interpreter when command is `auto` and no binary is on PATH
`pynescript.lsp.args`	`[]`	Extra args to the language server
`pynescript.formatting.enabled`	`true`	Format document
`pynescript.diagnostics.enabled`	`true`	Lint squiggles
`pynescript.completion.snippets`	`true`	Snippet completions

Extension id: `hoox-sh.pyne`.

Neovim (`clients/neovim.lua`):

```
settings = {
  pinescript = {
    formatting = { enabled = true },
    diagnostics = { enabled = true },
    completion = { snippets = true },
  },
}
```

Zed: `language_servers.pynescript.command` / `arguments: ["--stdio"]` — see Editors.

AXIS / frontend coupling (optional)

When running SuperChart Lite PWA against local Flask:

Make target	Port (typical)
`make run`	API `:5002`
`make run-frontend`	PWA `:8081`
`make worker-dev`	CF Worker `:8787`

CORS must allow the PWA origin via `ALLOWED_ORIGINS`.

Internals (repo paths)

Path	Config surface
`pyproject.toml`	extras, scripts, Python requires
`src/pynescript/__main__.py`	CLI options
`src/pynescript/runtime/host.py`	`Runtime.run` (`mode`, `libraries`, `timeout_seconds`, `inputs`)
`backend/app.py`	Flask, CORS, HOST/PORT, body size
`backend/middleware/auth.py`	`API_KEY_STORE`, key tiers
`backend/middleware/schemas.py`	Request field defaults (`mode`, `symbol`, ...)
`clients/*.json	lua
`vscode-extension/`	Extension contribution points

Invariants & edge cases

- **Unknown JSON fields on Pro API are rejected** (`UNKNOWN_FIELDS`) — schemas are strict.
- **CORS defaults exclude arbitrary production domains** — set `ALLOWED_ORIGINS` deliberately.
- **Key store default path is production-oriented** (`/root/...`) — always override locally.
- **Lint `--fail-on`** only affects process exit code; it does not change which rules run.
- **Recursion limit** during parse is raised to at least 5000 temporarily inside `helper._parse` for deep nests; not user-configurable.

Worked examples

Desk-only machine

```
python -m venv .venv && source .venv/bin/activate
pip install hoox-pyne
# no env vars required
pyne lint strategy.pine --fail-on errors
```

Editor workstation

```
pip install "hoox-pyne[lsp]"
# ensure which pyne-lsp  (alias: pynescript-lsp)
# paste clients/neovim.lua or clients/zed.json as documented
```

Local API + AXIS

```
pip install -e ".[lsp]"
pip install -r backend/requirements.txt

make run
# other terminal:
make run-frontend
```

Scripted Alpha Vantage fetch

```

pynescript data EUR/USD --provider alphavantage --api-key "$AV_KEY" --period 3mo
```

Failure modes

Symptom	Config fix
Browser CORS error on `/run`	Add origin to `ALLOWED_ORIGINS`
`create_key` always 403	Set `ADMIN_TOKEN` and send `X-Admin-Token`
API keys vanish / permission error	Point `API_KEY_STORE` to writable path
Payload too large	Split bars or stay under 5 MiB body limit
Editor “server failed to start”	Fix `pynescript.lsp.command` PATH; install `[lsp]`
`ccxt` import errors	Install `[data]` extra

See also

- Installation
- Pro API usage
- Editors
- CLI commands

- Runtime modes
- API auth (systems)
- AXIS docs

CLI

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

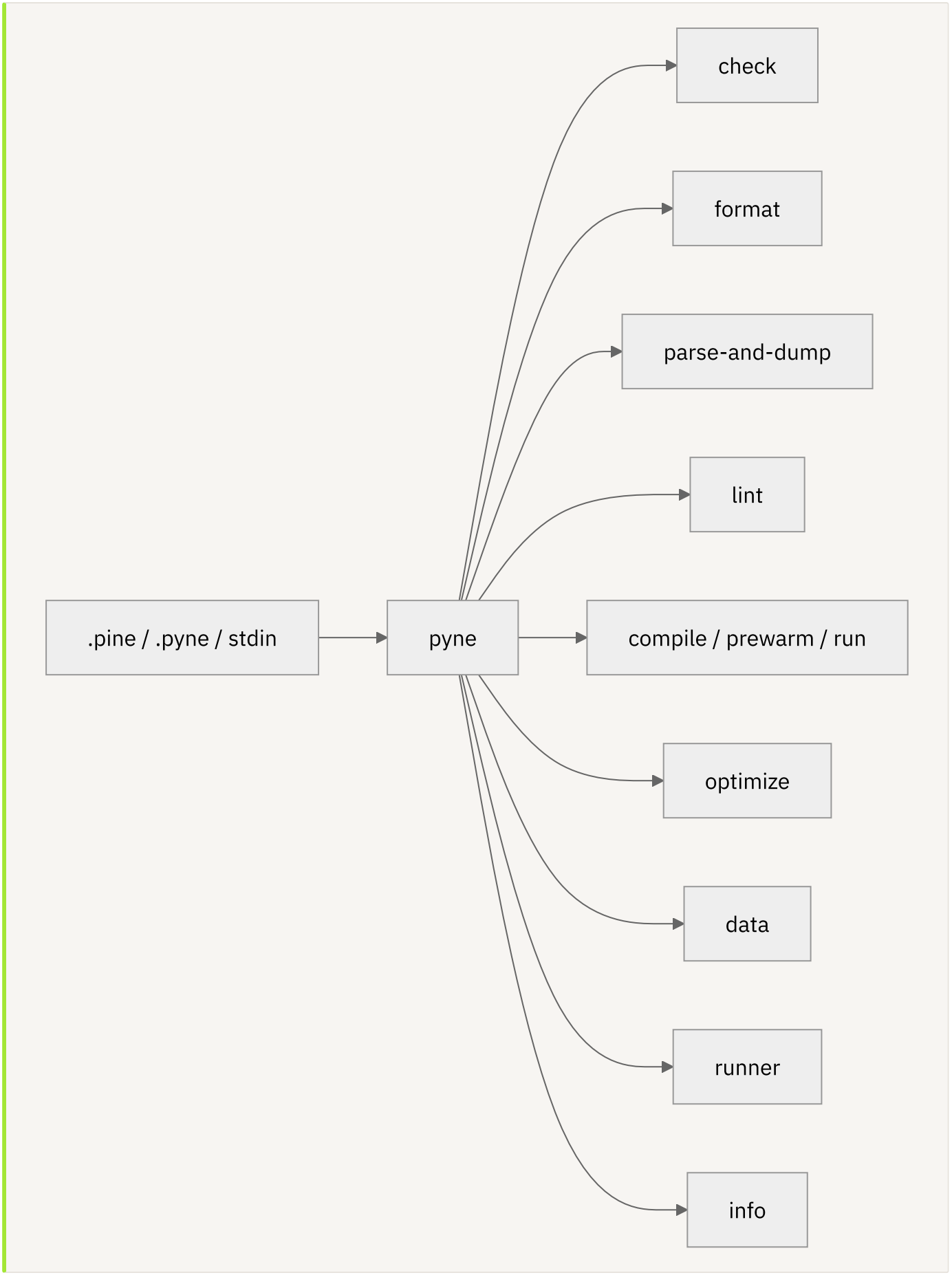
title: "CLI Guide" description: "Use the pyne Click CLI to check, format, lint, compile, run, optimize, prewarm, fetch data, and more."

CLI GUIDE

Abstract

The preferred `pyne` console script (alias `pynescript`) is a Click group over the same helpers used by the library API. Install with `pip install hoox-pyne` (live on PyPI). It is optimized for **shell pipelines, CI gates, one-shot inspection, and light compile/run smokes**. Full multi-bar evaluation with real OHLCV belongs on the package `pynescript.runtime.Runtime` or Pro API. Exhaustive flags: CLI commands.

Conceptual model



CLI	Library / subsystem
`check`	`parse` only (exit codes)
`format` / `fmt`	`parse` + `unparse`
`convert`	`pynescript.util.pine_convert` (any older version → v6)
`parse-and-dump` / `dump`	`parse` + `dump`
`lint`	`lint_script`
`compile` / `prewarm` / `run`	`pynescript.compiler` (+ `[compile]` extra)
`optimize`	`pynescript.optimize.run_study`
`data`	`pynescript.util.data.get_provider`
`runner`	HTTP client for Flask `/scripts` + `/cron/run` (`PYNE_RUNNER`)
`info`	version + optional extras

Interface surface

```
pyne --help
pyne --version
pyne <command> --help
# alias: pynescript …
```

Command	Input	Output
`check PATHS…`	files / dirs / stdin	ok/fail lines + exit code
`format PATH`	file or `-`	formatted source (`-w` / `--check`)
`convert PATH --to 6`	file or `-`	rewritten v6 source (`-w`)
`parse-and-dump PATH`	file	AST dump
`parse-and-unparse PATH`	file	Pine source
`lint [PATH|-]`	file or stdin	diagnostics (`--json`)
`compile PATH`	file	emit Python or compile-check
`prewarm [PATH…]`	optional files	warm Numba / IR cache
`run PATH`	file	synthetic-bar execute
`optimize PATH`	file + `--space` JSON	strategy hyperparameter search
`data SYMBOL`	symbol + provider opts	table / json / csv
`runner …`	file / id	hosted deploy + tick (runner)
`info`	—	version + extras

Not part of this Click group: language server — use **`pyne-lsp`** (alias `pynescript-lsp`).

Install options

Method	When
`pip install hoox-pyne`	Default desk / CI use
`pip install "hoox-pyne[compile]"`	Need Numba `compile` / `run` modes
GitHub Release binary `pynescript-cli-*`	No Python install
Docker `pynescript-cli`	Isolated CI / one-shot checks

```
# Docker
docker buildx bake cli
docker run --rm -v "$PWD:/work" -w /work pynescript-cli:latest check script.pine

# Compose (repo root)
make docker-cli ARGS="lint script.pine --json"

# Standalone binary (from GitHub Release)
chmod +x pynescript-cli-linux-x86_64
./pynescript-cli-linux-x86_64 info
```

Internals (repo paths)

Path	Role
`src/pynescript/__main__.py`	All command implementations
`src/pynescript/ast/helper.py`	`parse` , `dump` , `unparse`
`src/pynescript/ast/linter.py`	`lint_script`
`src/pynescript/compiler/`	`compile` / `prewarm` / `run`
`src/pynescript/util/data.py`	providers + `DataProviderError`
`src/pynescript/util/runner_cli.py`	`pyne runner` HTTP client
`examples/`	sample `.pine` inputs
`tests/test_cli.py`	CLI unit suite (CI)

Invariants & edge cases

- **Encoding default is UTF-8** for file reads/writes.
- **`--output-file -`** means stdout (Click `allow_dash=True`).
- **Lint without a path** reads stdin — useful in git hooks and `cat script.pine | pyne lint`.
- **`--fail-on`** defaults to `errors` (syntax/`E001`); `warnings` fails on any warning severity too; `all` fails if any issue exists.
- **`data` default provider is `mock`** — offline-safe.
- **Alpha Vantage without key** prints a warning and uses `demo`.
- **`lint` is one file or stdin** — no directory walk. Use `check` for recursive parse-only (`--ext`).
- **`run` is compile-only** on synthetic bars (`--bars` default 50). It does not call `Runtime.run` and has no `--mode`.

Worked examples

Inspect structure of a strategy

```
pyne parse-and-dump examples/rsi_strategy.pine --indent 2 | less
```

Compare two scripts' shapes in CI:

```
pyne parse-and-dump a.pine --output-file /tmp/a.ast
pyne parse-and-dump b.pine --output-file /tmp/b.ast
diff -u /tmp/a.ast /tmp/b.ast
```

Format / normalize

```
pyne format messy.pine -w
pyne format strategy.pine --check    # CI drift gate (exit 1 if would change)
pyne parse-and-unparse messy.pine --output-file clean.pine
```

Lint gate in CI

```
set -e
# lint is one file or stdin — not a directory walk
for f in strategies/*.pine strategies/*.pyne; do
  pyne lint --fail-on warnings "$f"
done
# parse-only gate over a tree:
pyne check strategies/ --ext .pine
```

Stdin pipeline:

```
git show HEAD:strategies/rsi.pine | pyne lint --fail-on errors -
```

Market data smoke checks

```
pyne data AAPL
pyne data AAPL --provider yahoo --period 1y --interval 1d
pyne data BTC/USDT --provider ccxt --exchange binance
pyne data EUR/USD --provider alphavantage --api-key "$AV_KEY"
```

Typical table keys (from `__main__.py`): `symbol`, `provider`, `bars`, `period / interval`, `first close`, `last close`, `change`, `high / low`, `avg volume`. `--format json` emits full OHLCV lists.

Compile, prewarm, synthetic run

```
pip install "hoox-pyne[compile]"
pyne compile script.pine              # compile-check (Numba or object-mode)
pyne compile script.pine --emit       # print generated Python
pyne prewarm                          # warm shared Numba kernels
pyne prewarm script.pine              # also compile that source into IR caches
pyne run script.pine --bars 100       # compile-only execute on synthetic OHLCV
```

Compose with library for evaluation

`pyne run` is compile-only smoke. Real bars + `mode` / `libraries` / `timeout_seconds` belong on the package Runtime:

```
pyne format strat.pine -o /tmp/s.pine
python - <<'PY'
from pathlib import Path
from pynescript.runtime import Runtime
# hand off to Runtime or your own visitor
print(Path("/tmp/s.pine").read_text()[:40])
PY
```

Failure modes

Exit / message	Cause	Mitigation
Click path errors	missing file	check `PATH` exists and is a file
Parse exception during dump	invalid syntax	fix source; use smaller repro
`Lint failed with errors.`	`--fail-on` threshold	read `E00x` / `W00x` lines
`DataProviderError` → ClickException	network / auth / missing dep	install `[data]`, fix keys
Empty dump / unexpected tree	`mode` always exec for CLI	expression-only needs library `mode="eval"`

See also

- CLI commands reference
- Runtime modes
- Library API
- Quick start
- Troubleshooting

LIBRARY API

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

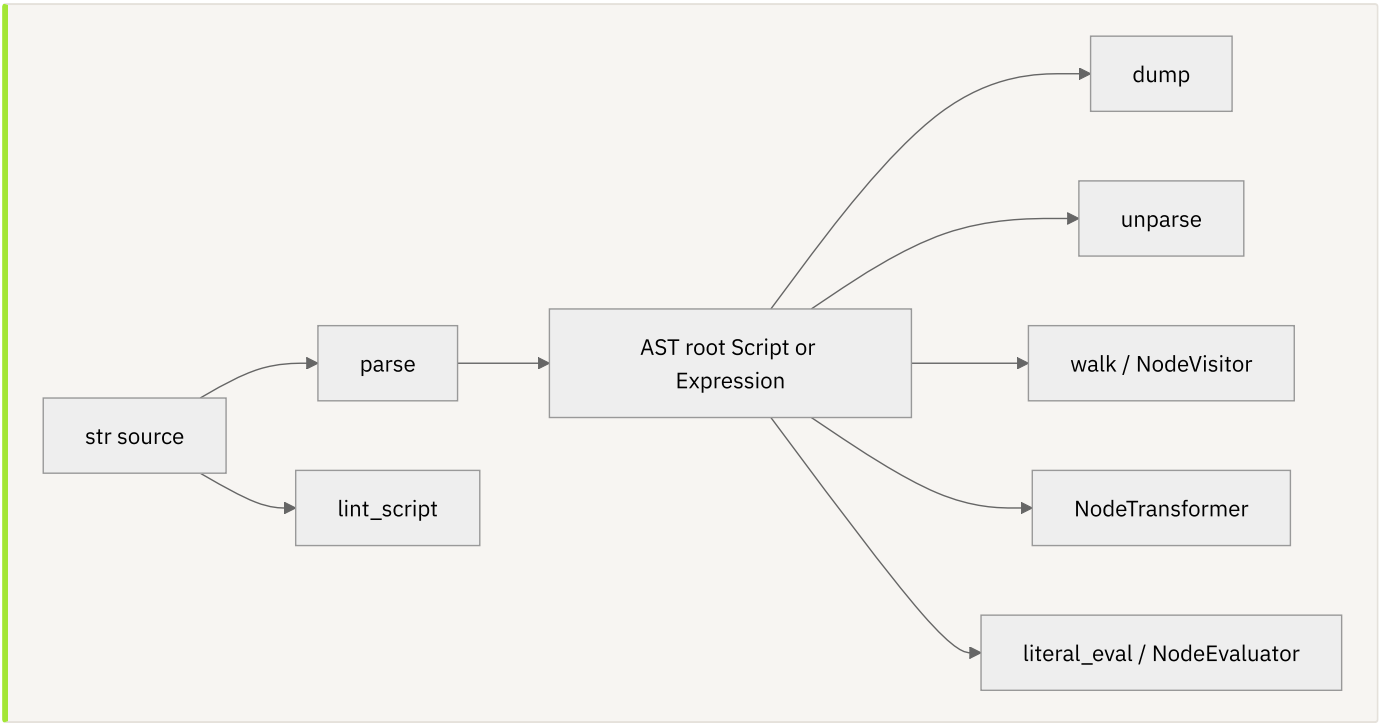
title: "Library API" description: "Embed pynescript: parse, dump, unparse, walk, transform, and lint Pine Script™ from Python."

LIBRARY API

Abstract

The public Python surface for language work lives primarily under `pynescript.ast`: parse source into an ASDL-backed tree, serialize it (`dump`), regenerate source (`unparse`), traverse (`walk`, visitors), rewrite (`NodeTransformer`), and lint (`lint_script`). Expression evaluation (`literal_eval`) and full script execution (`pynescript.runtime.Runtime`) sit adjacent; this page focuses on **structural** APIs. Evaluation workflows are covered in Evaluate scripts. Strategy hyperparameter search is `pynescript.optimize.run_study` (not a Pine builtin) — POST /optimize. Install via `pip install hoox-pyne` (`import pynescript`).

Conceptual model



Design intent mirrors Python's `ast` module: **stable tree operations**, location attributes, and visitor protocols — specialized for Pine's script model (annotations, series-aware semantics live at evaluation time).

Interface surface

Imports

```
# Re-exported convenience (node classes, helpers, visitors, transformers)
from pynescript.ast import parse, unparse, dump, walk
from pynescript.ast import NodeVisitor, NodeTransformer
from pynescript.ast.helper import literal_eval, iter_child_nodes, get_source_segment
from pynescript.ast.linter import lint_script, lint_file, PineLinter, LintWarning
```

`pynescript.ast.__init__` star-imports `helper`, `node`, `visitor`, `transformer`, and `error`.

`parse(source, filename="<unknown>", mode="exec") -> AST`

Arg	Meaning
`source`	Full script or expression text
`filename`	Error reporting; absolutized if file exists
`mode`	`"exec"` → script statements; `"eval"` → single expression

Raises `ValueError` on bad mode; `SyntaxError` (via error listener) on grammar failure.

Successful trees are cached in a process-local LRU (`sha256(source)` + `mode`). Disable with `PYNE_PARSE_CACHE=0`. Cached trees are **read-only** (shared by identity).

```
tree = parse('//@version=6\nindicator("x")\nplot(close)\n')
expr = parse("close > open", mode="eval")
```

`unparse(node) -> str`

Round-trips an AST through `NodeUnparser`.

```
normalized = unparse(parse(messy_source))
```

`dump(node, *, annotate_fields=True, include_attributes=False, indent=None) -> str`

`indent` may be an `int` (spaces per level) or a string. `None` is a single line.

Human-readable tree serialization (debug, tests, CLI).

```
print(dump(tree, indent=2, include_attributes=True))
```

`walk(node) -> Iterator[AST]`

Breadth-first (deque) traversal yielding every node.

```
from pynescript.ast import walk
names = [n for n in walk(tree) if n.__class__.__name__ == "Name"]
```

Location helpers

Function	Role
`copy_location(new, old)`	Copy lineno/col offsets
`fix_missing_locations(node)`	Fill missing positions
`increment_lineno(node, n=1)`	Shift lines
`get_source_segment(source, node, *, padded=False)`	Slice original text
`iter_fields` / `iter_child_nodes`	Schema-aware iteration

Visitors and transformers

```
from pynescript.ast import NodeVisitor, NodeTransformer, parse

class CallCounter(NodeVisitor):
    def __init__(self):
        self.calls = 0
    def visit_Call(self, node):
        self.calls += 1
        self.generic_visit(node)

class RenameClose(NodeTransformer):
    def visit_Name(self, node):
        # Name fields: id (identifier), ctx (Load | Store | …)
        if node.id == "close":
            node.id = "price"
        return node

tree = parse(source)
CallCounter().visit(tree)
tree2 = RenameClose().visit(tree)
```

Linter

```
from pynescript.ast.linter import lint_script, LintWarning

warnings: list[LintWarning] = lint_script(source, "strat.pine")
for w in warnings:
    print(w.code, w.severity, w.line, w.message)
```

Code family	Examples
`E001`	Syntax error (error severity)
`W001` – `W002`	Missing / old `@version`
`W101`	Deprecated `security('EXCHANGE:SYM')` (W102/W103 retired)
`C001` / `C002` / `C004`	Naming / style (line length 120, trailing newline). `C003` retired.

`PineLinter.lint` runs: syntax → version → deprecated → naming → style.

`literal_eval` (bridge to evaluation)

```
from pynescript.ast.helper import literal_eval

literal_eval("ta.sma([1,2,3,4,5], 3)")
literal_eval("close[1]", {"close": [10.0, 11.0, 12.0]})
```

Optional `data_feed` / `data_provider` for `request.*` in literal contexts.

Internals (repo paths)

Path	Role
`src/pynescript/ast/helper.py`	Public parse/dump/walk/unparse/literal_eval
`src/pynescript/ast/builder.py`	Parse tree → ASDL nodes
`src/pynescript/ast/unparser.py`	`NodeUnparser`
`src/pynescript/ast/visitor.py`	`NodeVisitor`
`src/pynescript/ast/transformer.py`	`NodeTransformer`
`src/pynescript/ast/node.py`	Node re-exports / helpers
`src/pynescript/ast/grammar/asdl/resource/Pinescript.asdl`	Algebraic schema
`src/pynescript/ast/linter.py`	Static rules
`src/pynescript/ast/error.py`	Error types
`examples/parse_dump_unparse.py`	Round-trip demo

Pipeline inside `parse`:

1. `InputStream` / `FileStream`
2. `PinescriptLexer` + `PinescriptParser` (error listener)
3. `PinescriptASTBuilder.visit`
4. In `exec` mode: `StatementCollector`, comment collection, `@` annotation attach

Invariants & edge cases

- **Annotation comments** (`//@version`, etc.) attach to script / function / type / assign nodes by kind suffix — see `_add_annotations` in `helper.py`.
- **Deep nesting:** temporary `sys.setrecursionlimit(max(old, 5000))` during parse.
- **`dump` requires a true AST node** — `TypeError` otherwise.
- **Transformers mutate or replace nodes** depending on your visit methods; prefer returning new nodes if you need purity.
- **Round-trip fidelity** is tested against a large corpus but is not a proof of bit-identical source; whitespace and some sugar may normalize.
- **Do not edit generated grammar modules**; extend via resource grammars if contributing.

Worked examples

Strategy hyperparameter search (0.3.12)

```
from pynescript.optimize import run_study, space_from_payload
from pynescript.optimize.types import ValidationSpec

space = space_from_payload({
    "params": [{"name": "Fast", "kind": "int", "min": 3, "max": 12}],
})
study = run_study(
    open("strategy.pine").read(),
    ohlcv_bars,
    space,
    n_trials=20,
    sampler="tpe",
    validation=ValidationSpec(mode="holdout"),
)
print(study.best_params, study.best_oos_score)
```

HTTP: POST /optimize. CLI: `pyne optimize`.

End-to-end inspect

```
from pynescript.ast import parse, dump, unparse, walk

source = open("examples/rsi_strategy.pine", encoding="utf-8").read()
tree = parse(source, "examples/rsi_strategy.pine")
print(dump(tree, indent=2)[:500], "...")
print("---")
print(unparse(tree))
print("node count", sum(1 for _ in walk(tree)))
```

Collect all call names

```
from pynescript.ast import parse, NodeVisitor

class Calls(NodeVisitor):
    def __init__(self):
        self.names = []
    def visit_Call(self, node):
        func = node.func
        # Attribute vs Name depending on ta.rsi vs f()
        self.names.append(func)
        self.generic_visit(node)

c = Calls()
c.visit(parse(source))
```

Lint programmatically with fail semantics

```
from pynescript.ast.linter import lint_script

def assert_clean(path: str) -> None:
    text = open(path, encoding="utf-8").read()
    issues = lint_script(text, path)
    errors = [i for i in issues if i.severity == "error"]
    if errors:
        raise SystemExit("\n".join(map(str, errors)))
```

Expression mode for tooling

```
from pynescript.ast import parse, dump

print(dump(parse("ta.rsi(close, 14)", mode="eval"), indent=2))
```

Failure modes

Exception / result	Meaning
`SyntaxError`	Lexer/parser rejection
`ValueError: invalid argument mode`	mode not in `{exec,eval}`
Empty / odd tree	Empty body scripts; annotations-only edge cases
Unparse mismatch	Node kinds without unparser support — file issue against unparser
Linter false positives	Style rules are heuristic (regex naming) — treat `C00x` as advisory

See also

- Evaluate scripts
- CLI guide
- Helper API (core)
- Visitor / transformer (core)
- Linter (core)

EVALUATE SCRIPTS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Evaluate Scripts" description: "Run Pine expressions and multi-bar scripts with literal_eval, pynescript.runtime.Runtime (libraries=, timeout_seconds, mode), data providers, and strategy events."

EVALUATE SCRIPTS

Abstract

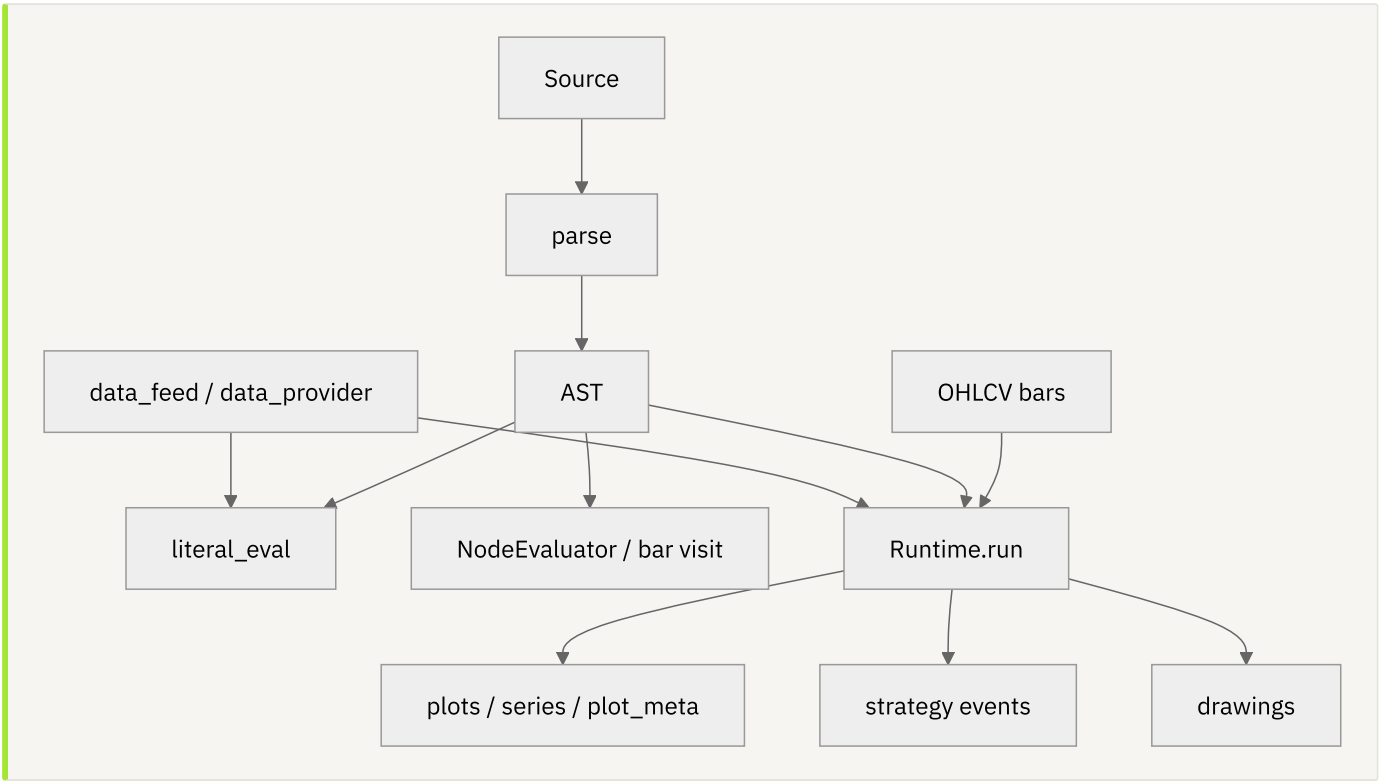
Evaluation is the second half of the PYNE pipeline: given an AST (or source) and **market context**, produce plots, series, strategy events, and drawings. End users typically choose among three graded tools:

1. `literal_eval` — pure/builtin expressions, optional series context
2. **AST evaluator** (`NodeLiteralEvaluator` / full evaluator mixins) — script-shaped evaluation in-process

3. **`pynescript.runtime.Runtime`** (package SoT; `backend.runtime` re-exports) or **Pro API** **`POST /run`** — bar-loop over OHLCV with shared evaluate contract. TypeScript: **`@hoox-sh/pynets`** **`Runtime.run`** — PyneTS. Defaults differ by surface — modes.

This guide stays at the consumer level; series semantics and builtin inventories are under Runtime.

Conceptual model



Mode	Bar loop	Strategy	Typical use
`literal_eval`	no (single shot)	limited	TA on arrays, math, strings
Evaluator in tests / tools	optional	yes via state	Unit tests, notebooks
`Runtime.run` / `/run`	yes	yes	Charts, AXIS, backtests
`mode="compile"`	yes (numeric + object)	yes (object mode)	Faster bar loop when eligible
`mode="auto"`	yes	yes	Prefer compile; fall back to interpret

Runtime modes (`interpret` | `compile` | `auto`)

`pynescript.runtime.Runtime.run` accepts a `mode` argument that selects how the bar loop executes:

Mode	Behavior
`interpret`	AST walker (full host semantics: `input.*` overrides, `request.*`, libraries, profiler).
`compile`	Transpile to a Numba numeric kernel when possible, otherwise a pure-Python **object-mode** bar loop. Requires the compiler package.
`auto`	Try compile first; on eligibility failure, compile error, or compiled runtime error, **fall back to interpret**.

Defaults differ by surface:

- Direct `Runtime.run(..., mode=None)` → `PYNE_RUNTIME_MODE` env, else **`interpret`**.
- Pro API `POST /run` body omit `mode` → schema default **`auto`** (warm compile + fallback).

```
from pynescript.runtime import Runtime

runtime = Runtime(symbol="AAPL")
result = runtime.run(
    source,
    ohlcv_data=ohlcv,
    mode="auto",                # omit → PYNE_RUNTIME_MODE else "interpret"
    timeout_seconds=12.0,       # interpret wall-clock budget; None = no limit
    libraries=[                 # import ns/Name/ver (max 32 on POST /run)
        {"namespace": "ns", "name": "Lib", "version": 1, "source": '//@version=6\nlibrary("Lib")\ne
    ],
    inputs={"len": 14},         # input.* by title — forces interpret under auto
)
# result["mode"] / result.get("auto_backend") → "compile" | "interpret"
# on fallback: result.get("compile_fallback_reason")
# on timeout: timed_out + error_kind="runtime" with partial plots
```

`timeout_seconds` is accepted on the library host and on `POST /run` / `POST /run/batch` (omit / `null` / `≤ 0` → no budget). `pyne run` does not call `Runtime.run` at all.

Warm compile / prewarm

Product hosts cache compiled IR (in-process LRU + optional disk under `PYNE_COMPILE_CACHE_DIR`). Pro API defaults `mode=auto` and can warm Numba builtins once per worker (`PYNE_COMPILE_PREWARM`, `POST /compile/prewarm`, or `pyne prewarm [PATH…]`). First-hit latency still pays JIT when cold; subsequent runs of the same source reuse IR. See Compiler overview.

Related host knobs (interpret path): `PYNE_SERIES_CAP` / `PYNE_SERIES_MAX` (history trim) and `PYNE_TA_INCREMENTAL` (bar-mode TA hot path)—see Series & history and Configuration.

From **0.3.10**, interpret also skips unused derived OHLCV (`hl2` / `hlc3` / `ohlc4` / `tr`) and inlines Assign / plot / Call after bar 0. Same-machine bench @ 2000 bars vs 0.3.9: minimal **2.7×**, `ta.sma` **2.0×**, multi-plot TA combo **1.45×**. Prefer `mode="auto"` on Pro `/run` when the script is compile-eligible; interpret remains the full-surface oracle.

When compile falls back (or is skipped)

`mode="auto"` (and explicit prefilters) skip or abandon compile for stable reasons. Common `compile_fallback_reason` values:

Reason (examples)	Why
`input.* overrides require interpret path`	Non-empty `inputs=` are interpret-only; compile does not apply overrides.
`import statements not supported in compile path`	Library `import` needs the interpreter registry.
`request.* not supported in compile path`	Multi-symbol / external data plumbing is interpret-only.
`compiler package unavailable`	`pynescript.compiler` not importable in this install.
`Compile Error: …` / Numba required messages	Deterministic transpile/env failures (cached per source for auto).
Compiled runtime errors	Data-dependent failures still fall back once; not permanently cached as "never compile."

Other force-interpret paths:

- **`profiler=True`** — line timings need the AST walker; compile/auto are coerced to interpret.
- **`mode="compile"`** (strict) — does **not** fall back; you get an error payload instead of a silent interpret run.

Numba is **not** required for compile eligibility: strategy / UDT / map-heavy scripts can still run in object-mode compile. Missing Numba only blocks pure-numeric emit.

Plot series parity

Both backends export the same Runtime contract shape: `series` (title → per-bar values), `plots`, `plot_meta`, `events`, `drawings`, plus `mode` / diagnostics. For charting, **shared plot series keys** should agree within floating-point tolerance (`na`/None/NaN treated as NA).

First-party `hline` / `fill` / `bgcolor` / `plotshape` **keys match** interpret ↔ compile (0.3.11). **0.3.12** also matches `barcolor` / `plotarrow` / `plotbar` / `plotcandle` kinds and geometry-only `drawings` (line/box/label/polyline/table/linefill) — coordinates are snapshotted at create/set. Harness `--ignore-hline-keys` / `--ignore-fill-keys` stay optional for leftover **corpus** noise, not first-party fixtures. Deeper compiler notes: Compiler overview.

Compare interpret vs compile (corpus harness)

From the repo root (monorepo with `backend/` + editable `pynescript`):

```
# Smoke: first 50 corpus scripts × 1000 synthetic bars
python scripts/compare_interp_compile.py --bars 1000 --limit 50

# Full list from a file, no script limit, 4 workers, 30s per script;
# ignore one-sided hline/fill keys when judging value parity
python scripts/compare_interp_compile.py --file-list path.txt --limit 0 --workers 4 --timeout-sec 3
```

The harness runs each script with `mode="interpret"` and `mode="compile"`, then nan-aware `allclose` on common `result["series"]` keys (`rtol=1e-5`, `atol=1e-6`). Report: `.cache/interp_compile_parity.json`. Exit 0 when there are no value/NaN mismatches on shared keys (`both_error_same` counts as success unless `--strict-errors`).

Interface surface

Expression evaluation

```
from pynescript.ast.helper import literal_eval

literal_eval("1 + 2 * 3")
literal_eval("math.max(1, 5, 3)")
literal_eval('str.upper("hi")')
literal_eval("array.size([1, 2, 3])")

prices = [100, 102, 101, 103, 105, 104, 106, 108, 107, 110]
literal_eval(f"ta.sma({prices}, 5)")
literal_eval(f"ta.rsi({prices}, 9)")
bb = literal_eval(f"ta.bb({prices}, 5, 2)")  # middle, upper, lower
```

Series history context (Pine-style `[0]` current / `[1]` previous as implemented):

```
context = {
    "close": [100, 102, 101, 103, 105],
    "open": [99, 101, 100, 102, 104],
    "high": [101, 103, 102, 104, 106],
    "low": [98, 100, 99, 101, 103],
}
literal_eval("close[0]", context)
literal_eval("close[0] - close[1]", context)
```

Optional live/historical wiring:

```
literal_eval(expr, context, data_feed=feed, data_provider=provider)
```

Script evaluation helper

```
from pynescript.ast.evaluator import NodeLiteralEvaluator

ev = NodeLiteralEvaluator()
result = ev.evaluate_script(
    """
//@version=6
indicator("demo")
// body depends on what the evaluator implements for statements
"""
)
```

Libraries:

```
ev.register_library_source(namespace="MyNs", name="Lib", version=1, source=lib_source)
mod = ev.lookup_library(namespace="MyNs", name="Lib", version=1)
```

Bar-loop Runtime (package SoT · HTTP contract shape)

From **0.3.4** the bar-loop host lives in the installable package:

```
from pynescript.runtime import Runtime  # SoT (backend.runtime re-exports for Pro API)

runtime = Runtime(symbol="AAPL")
ohlcv = [
    {"time": 1_700_000_000, "open": 100, "high": 101, "low": 99, "close": 100.5, "volume": 1_000},
    {"time": 1_700_086_400, "open": 100.5, "high": 102, "low": 100, "close": 101.5, "volume": 1_200
]
result = runtime.run(
    source_code='//@version=6\nindicator("t")\nplot(close)\n',
    ohlcv_data=ohlcv,
    mode="auto",  # interpret | compile | auto (see Runtime modes)
    timeout_seconds=12.0,
    # libraries=[{"namespace": "ns", "name": "Lib", "version": 1, "source": lib_src}],
)
# result: plots, series, plot_meta, events, drawings, script_id, run_id, count, ...
# auto may set auto_backend + compile_fallback_reason
# or {"error": "...", "error_kind": "parse|compile|runtime|data|order|mode", ...}
```

`backend.runtime.Runtime` remains a thin compat import for monorepo Pro API code. Dual-host TA goldens (ATR / Supertrend / Keltner) and `tests/test_ta_incremental` gate interpret↔compile numerical parity. Pro API wraps the same runtime — see Pro API usage.

Persistent interpret sessions (0.6.1)

`Runtime.run` always replays the full history. For live or incremental feeds, **`Runtime.create_session`** keeps the evaluator, series, TA state, and plot columns across calls so new closed bars only pay `visit(tree)` for the delta:

```
from pynescript.runtime import Runtime

rt = Runtime(symbol="AAPL")
session = rt.create_session(source, ohlcv_data=bars)  # interpret only
result = session.append_bars(new_closed_bars)         # O(delta)
tick = session.update_last_bar(forming_bar)           # overwrite last bar in place
```

Method	Role
`create_session(source, ohlcv, …)`	Parse once, evaluate the initial bars, return an `InterpretSession`. `mode` must be omitted or `"interpret"` (compile has no resumable evaluator state).
`append_bars(new_bars, realtime_ticks=1, …)`	Confirm the previous last bar, then visit only the appended batch. Realtime window params apply to **this batch**, not previously committed bars.
`update_last_bar(bar)`	Re-tick the forming last bar with revised OHLCV. Bar count does not advance. `var` scope and incremental TA windows roll back to bar-open (`varip` persists).

Streaming contract: already-committed bars keep their values (like TradingView® confirmed history). Scripts that branch on `barstate.islast` will diverge from a full `Runtime.run` replay for the previously last bar — that bar was committed with `islast=True`. Prefer `islastconfirmedhistory` / historical logic for append-parity, or re-run full `Runtime.run` when exact `islast` replay is required. `PYNE_SERIES_RING` must stay off (the default). Tests: `tests/test_runtime_session.py`.

Data providers and feeds

Historical CLI/library:

```
from pynescript.util.data import get_provider

prov = get_provider("mock")
# or yahoo / alphavantage / ccxt with kwargs
bars = prov.fetch("AAPL", "6mo", "1d")
# bars: dict with close/open/... lists
```

Realtime (requires ccxt / pro):

```
# examples/realtime_datafeed.py
from pynescript.util.datafeed import get_datafeed

feed = get_datafeed("ccxtpro", exchange="binance")
# async with feed: async for candle in feed.watch_ohlcv("BTC/USDT", "1m"): ...
```

Educational bar executor

`examples/execute_script.py` implements a **teaching** RSI strategy executor with a custom visitor and pandas history (`examples/historical_data.py` / yfinance). It is not the production `Runtime`, but demonstrates bar iteration patterns.

Internals (repo paths)

Path	Role
`src/pynescript/ast/helper.py`	`literal_eval`
`src/pynescript/ast/evaluator/base.py`	Context, series, visitor base
`src/pynescript/ast/evaluator/__init__.py`	`NodeLiteralEvaluator` composition
`src/pynescript/ast/evaluator/builtins/`	`ta.*`, `strategy.*`, arrays, …
`src/pynescript/ast/evaluator/events.py`	Event emission hooks
`src/pynescript/runtime/host.py`	Multi-bar `Runtime.run` SoT (`interpret` / `compile` / `auto` , `timeout_seconds`) plus `InterpretSession` (`create_session` / `append_bars` / `update_last_bar`)
`src/pynescript/runtime/series.py`	`PineSeries` history model
`backend/runtime.py` / `backend/series.py`	Compat re-exports of package Runtime
`src/pynescript/compiler/`	Transpile + Numba/object bar loops
`src/pynescript/util/data.py`	Providers + `resolve_request_sources`
`src/pynescript/util/datafeed.py`	Realtime feeds
`scripts/compare_interp_compile.py`	Interpret vs compile series parity harness
`tests/test_interp_compile_parity.py`	Always-on smoke + optional full mark
`examples/evaluate_expressions.py`	Expression gallery
`examples/execute_script.py`	Didactic strategy loop
`examples/rsi_strategy.pine`	Sample strategy source

Invariants & edge cases

- **Determinism.** Same source + same OHLCV + same mode should yield reproducible series (mock providers seedable via options where supported).
- **`na` / warmup.** TA functions need enough bars; early bars may be `na` /None-like — guard like Pine (`not na(x)`).
- **`mode="compile"`** uses numeric Numba when the script is pure-numeric; otherwise **object-mode** compile. Strict `compile` errors instead of falling back; use **`auto`** for production “prefer fast path” behavior (see Runtime modes).
- **`mode="auto"`** may set `compile_fallback_reason` and still return a successful interpret result — check `auto_backend` if you need to know which path ran.

- **Series parity.** Shared `series` keys should match within harness tolerances. First-party `hline` / `fill` / `bgcolor` / `plotshape` / `barcolor` / `plotarrow` / `plotbar` / `plotcandle` keys match; `drawings` is geometry-only. Ignore flags are for leftover corpus noise.
- **History indexing.** Confirm `[0]` / `[1]` semantics against your evaluator version before porting TV scripts that assume closed-bar rules.
- **Strategy events** include run/script ids when stamped by Runtime — useful for HOOX downstream.
- **Body size / bar count.** HTTP path caps payload size (5 MiB); very long histories should be downsampled or chunked.
- **InterpretSession.** `create_session` is interpret-only. `append_bars` confirms the previous last bar; scripts that branch on `barstate.islast` are not replay-identical to a full `Runtime.run` for that bar. `PYNE_SERIES_RING` must stay off.

Worked examples

RSI on a synthetic series

```
from pynescript.ast.helper import literal_eval

closes = [float(x) for x in range(100, 130)]
print(literal_eval(f"ta.rsi({closes}, 14)"))
```

Multi-indicator expressions

```
highs = [c + 1 for c in closes]
lows = [c - 1 for c in closes]
macd, signal, hist = literal_eval(f"ta.macd({closes}, 12, 26, 9)")
atr = literal_eval(f"ta.atr({highs}, {lows}, {closes}, 14)")
```

Full script via Runtime

```
from pathlib import Path
from pynescript.runtime import Runtime

script = Path("examples/rsi_strategy.pine").read_text(encoding="utf-8")
# Build ohlcv list from your provider...
rt = Runtime(symbol="EXAMPLE")
out = rt.run(script, ohlcv_data=ohlcv, mode="interpret")
if "error" in out:
    raise RuntimeError(out["error"])
print(out.get("count"), len(out.get("events", [])), out.get("series", {}).keys())
```

Strategy event inspection

```
for ev in out.get("events", []):
    # shape depends on StrategyState / event schema
    print(ev)
```

request.* with resolved sources

When calling `/run` or Runtime, pass `data_source` / providers so `request.security` and friends resolve; without configuration they may use chart bars or mocks.

Failure modes

Symptom	Interpretation
`NotImplementedError` / incomplete builtin	Expression outside supported surface — check missing features
`{"error": "Parse Error: ..."}`	Runtime caught parse failure
`{"error": …}` with `mode="compile"`	Strict compile path failed — retry with `auto` or `interpret`, or fix unsupported construct
`compile_fallback_reason` set under `auto`	Compile skipped/failed; result is from interpret if no top-level `error`
`EXECUTION_ERROR` via HTTP	Exception during bar loop
Empty plots	No `plot` / `plotshape` executed; wrong declaration type
Numerical mismatch vs TV	Warmup, float path, or builtin parity gap — numerical validation
Interpret vs compile series MISMATCH	Run `scripts/compare_interp_compile.py` on the script; check report buckets
`timed_out` + partial plots	`timeout_seconds` budget exceeded on interpret
`create_session supports interpret only`	`mode="compile"` / `"auto"` passed to `create_session` — sessions are interpret-only
`UNKNOWN_FIELDS`	Extra key not in `RUN_SCHEMA` / `RUN_BATCH_SCHEMA`
`DataProviderError`	Provider misconfig
Async feed errors	Missing ccxt.pro / network

See also

- Pro API usage
- Runtime modes
- Library API
- Runtime series model
- Compiler overview
- Strategy builtins
- Evaluate contract
- AXIS — visualizing series
- HOOX — consuming strategy events on the edge

EDITORS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

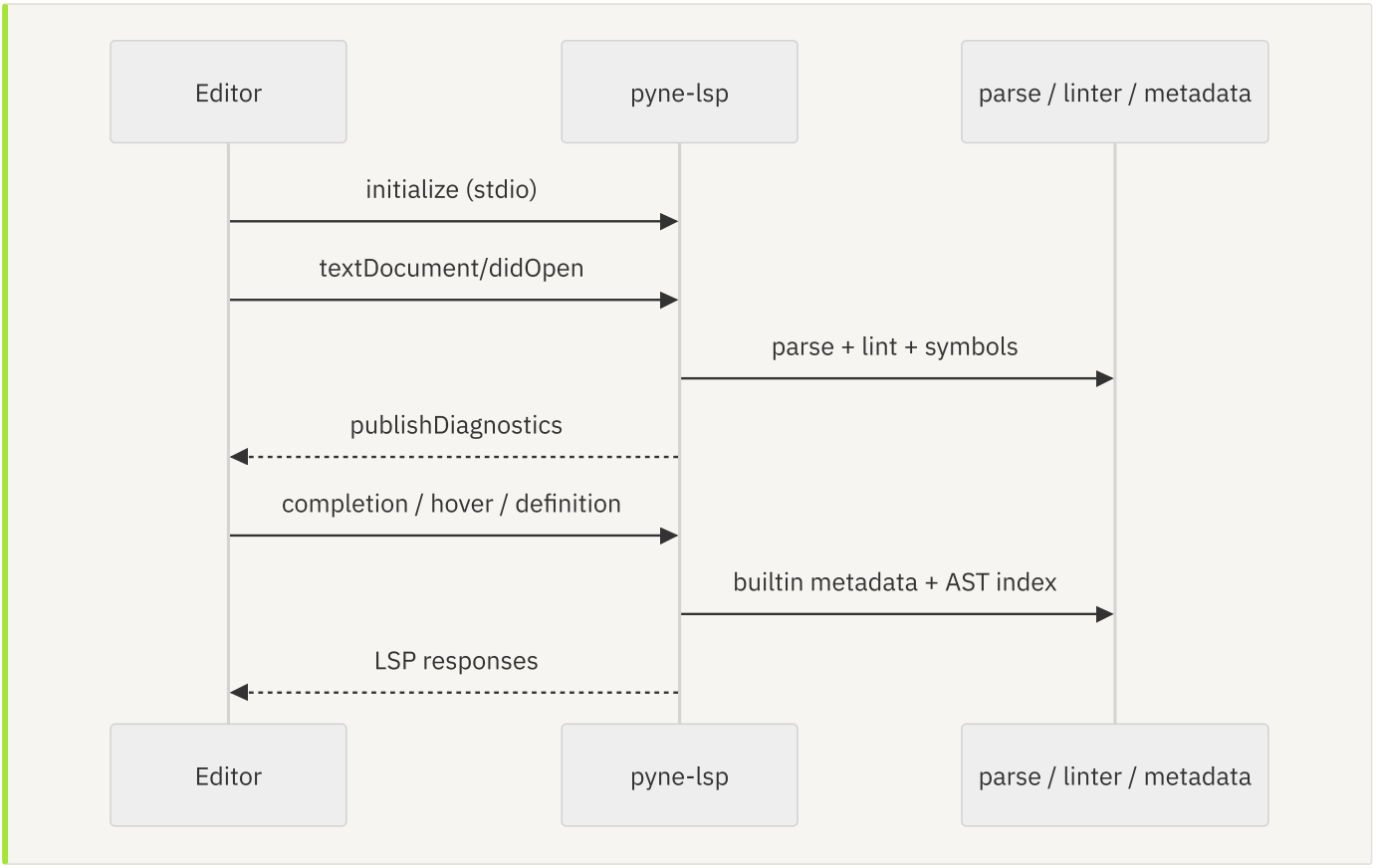
title: "Editors & LSP" description: "Wire pyne-lsp into VS Code, Neovim, Zed, Emacs, Helix, and other LSP clients for diagnostics, completion, hover, and formatting."

EDITORS & LSP

Abstract

`pyne-lsp` (alias `pynescript-lsp`) is a **stdio Language Server** (pygls) over the same parse/lint/metadata stack as the desk tools. Editors do not reimplement Pine Script™ intelligence; they speak LSP. This guide covers install, client configs in `clients/`, and feature expectations. Server architecture is documented under LSP.

Conceptual model



Feature	Role
Diagnostics	Lint rules + parse errors as squiggles
Completion	Builtins (`ta.*` , `strategy.*` , …) + snippets
Hover	Signature / docs from metadata
Definition / references	Symbol index in document
Document symbols	Outline / breadcrumb
Formatting	Full document + range via unparser path

Builtin metadata is generated (`scripts/generate_builtin_metadata.py`); encrypted blobs may appear in release builds.

Interface surface

Install server

```
pip install "hoox-pyne[lsp]"
# monorepo:
pip install -e ".[lsp]"
# or: make install

command -v pyne-lsp          # preferred
command -v pynescript-lsp    # alias
python -m pynescript.langserver  # equivalent entry
```

The process speaks **LSP over stdio** by default (`PynescriptLanguageServer.start_io()`). Do not expect a useful interactive TTY UI.

VS Code / Cursor / Antigravity

1. Build or install the extension from `vscode-extension/` :

```
cd vscode-extension
npm ci
npm run compile
# package: npx vsce package  (or make build-vscode)
```

2. Open a `.pyne` (preferred for HOOX / PYNE stack sources), `.pine` , `.pinev5` , `.pinev6` , or `.pinescript` file — LSP activates when enabled. The VS Code extension also activates on `workspaceContains:**/*.{pyne,pine,…}` .

Marketplace / VSIX: extension id `hoox-sh.pyne` (publisher `hoox-sh` , package name `pyne`). VS Marketplace. Site download: hoox.sh/pyne-vscode.vsix (unversioned alias). Local package: `cd vscode-extension && npm ci && npm run package` then `code --install-extension pyne-vscode-0.4.4.vsix` . The VSIX is self-contained; you still need `pip install "hoox-pyne[lsp]"` (or a `pyne-lsp` binary) on the machine.

Settings (from `vscode-extension/package.json`):

Key	Default	Notes
`pynescript.lsp.enabled`	`true`	Master switch
`pynescript.lsp.command`	`auto`	`auto` → `pyne-lsp` → `pynescript-lsp` → `python3 -m pynescript.langserver` ; or an absolute path / `docker`
`pynescript.lsp.python`	`python3`	Interpreter when command is `auto` and no binary is on PATH
`pynescript.lsp.args`	`[]`	Extra server args
`pynescript.formatting.enabled`	`true`	
`pynescript.diagnostics.enabled`	`true`	
`pynescript.completion.snippets`	`true`	

Default formatter id: `hoox-sh.pyne` .

Neovim (nvim-lspconfig)

```
require("lspconfig").pynescript.setup({})
```

Manual config from `clients/neovim.lua` :

```
return {
  cmd = { "pyne-lsp" },  -- alias: pynescript-lsp
  filetypes = { "pinescript" },
  root_dir = function(fname)
    return vim.fs.root(fname, { ".git", "*.pyne", "*.pine", "*.pinev5", "*.pinev6", "pyproject.toml
      or vim.fn.getcwd()
  end,
  settings = {
    pinescript = {
      formatting = { enabled = true },
      diagnostics = { enabled = true },
      completion = { snippets = true },
    },
  },
}
```

Suggested maps: `gd` definition, `gr` references, `K` hover, format via `vim.lsp.buf.format`.

Ensure filetype detection maps `*.pyne` / `*.pine` → `pinescript` if your distro does not ship it.

Zed

Merge `clients/zed.json` or:

```
{
  "languages": {
    "Pine Script": {
      "language_servers": ["pynescript"]
    }
  },
  "language_servers": {
    "pynescript": {
      "command": "pyne-lsp",
      "arguments": ["--stdio"]
    }
  }
}
```

Emacs (lsp-mode)

```
(use-package lsp-mode
  :hook ((pinescript-mode . lsp))
  :config
  (lsp-register-client
   (make-lsp-client
    :new-connection (lsp-stdio-connection '("pyne-lsp" "--stdio"))
    :major-modes '(pinescript-mode)
    :server-id 'pynescript)))
```

Or `(load-file "clients/emacs.el")` from a checkout.

Helix

`~/.config/helix/languages.toml`:

```
[[language]]
name = "pinescript"
scope = "source.pinescript"
file-types = ["pyne", "pine", "pinev5", "pinev6"]
roots = ["pyproject.toml"]
command = "pyne-lsp"
args = ["--stdio"]
```

Sublime Text (LSP package)

```
{
  "clients": {
    "pynescript": {
      "command": ["pyne-lsp", "--stdio"],
      "selector": "source.pinescript",
      "initializationOptions": {}
    }
  }
}
```

Generic

Any LSP client:

```
Command: pyne-lsp          # alias: pynescript-lsp
Args:    (none or --stdio depending on client)
Transport: stdio
```

Internals (repo paths)

Path	Role
`src/pynescript/langserver/__main__.py`	CLI entry `main()`
`src/pynescript/langserver/server.py`	`PynescriptLanguageServer`
`src/pynescript/langserver/features/`	diagnostics, completion, hover, ...
`clients/README.md`	Canonical client matrix
`clients/neovim.lua`, `zed.json`, `emacs.el`	Drop-in snippets
`vscode-extension/`	VS Code extension
`scripts/generate_builtin_metadata.py`	Completion/hover corpus

Invariants & edge cases

- **Python 3.10+** required on the machine running the server.
- **Server must be on PATH** or configured with absolute command.
- **One server per workspace** is typical; large monorepos with many `.pine` files still parse on demand.
- **Formatting** uses the unparser pipeline — expect normalization, not clang-format style knobs.
- **Diagnostics** reuse linter codes (`E001`, `W001`, ...); severity maps to LSP DiagnosticSeverity.

- **Nuitka binary** (`dist/lsp/pynescript-lsp`) may replace the Python entry in packaged distributions — point the editor at that binary if you ship it.

Worked examples

Verify server starts (stdio smoke)

```
# Editors start this; manual smoke is limited.
python -c "from pynescript.langserver.server import PynescriptLanguageServer; print(PynescriptLangu
```

Neovim minimal `init.lua` fragment

```
vim.api.nvim_create_autocmd("FileType", {
  pattern = "pinescript",
  callback = function()
    vim.lsp.start({
      name = "pynescript",
      cmd = { "pyne-lsp" },
      root_dir = vim.fn.getcwd(),
    })
  end,
})
```

Force a specific venv binary in VS Code

```
{
  "pynescript.lsp.command": "/home/you/project/.venv/bin/pyne-lsp"
}
```

Failure modes

Symptom	Fix
Client: executable not found	Install `[lsp]`; fix PATH / command setting
`No module named pygls`	Reinstall extra in the same env as the command
No diagnostics	Enable diagnostics; confirm filetype; check server logs
Stale completions	Regenerate metadata in dev builds; restart server
Format does nothing	Enable formatting; ensure document is valid enough to parse
High CPU on huge files	Split libraries; wait for parse; check for pathological nesting

See also

- Installation
- LSP architecture
- VS Code extension
- LSP clients (systems)

- Troubleshooting

PRO API USAGE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Pro API Usage" description: "Call the PYNE Flask Pro API as a consumer: health, /run, /run/batch, /optimize, chart preview, indicator preview, quick backtest, and auth."

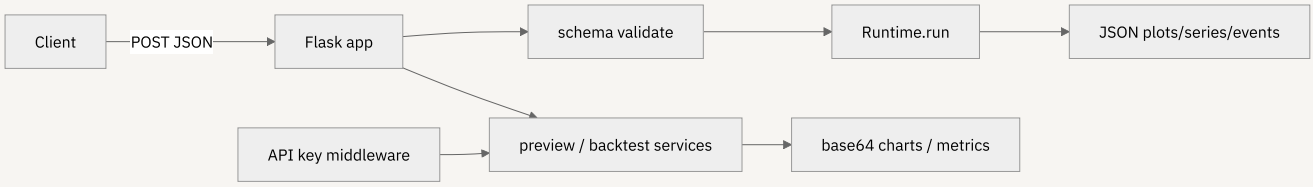
PRO API USAGE

Abstract

The Pro API is the **HTTP evaluate contract** for PYNE: POST Pine source plus OHLCV, receive plots, series, events, drawings, and **alerts**. Free-tier evaluate endpoints (`/run` , `/run/batch`) are usable without a key. Bar/script/rate/concurrency and chart/mock-only data guards apply **only when** `FREE_TIER_LIMITS` is truthy (`1` / `true` / `yes` / `on`); default **off**. Production compose sets `${FREE_TIER_LIMITS:-1}` . Webhook URLs stay SSRF-

safe regardless. Optional **L2 alert webhooks** POST last-bar firings to `webhook_url` or `ALERT_WEBHOOK_URL`. Pro preview/backtest routes track usage via API keys. This page is for **callers** (AXIS, scripts, workers). Server lifecycle and auth design are under API.

Conceptual model



Endpoint	Auth	Purpose
`GET /` / `GET /health`	none	Health + endpoint map (`default_run_mode` : `auto`)
`POST /run`	none (free)	Single script evaluate
`POST /run/batch`	none (free)	≤8 scripts, shared OHLCV
`POST /optimize`	none (free)	Strategy hyperparameter search (docs)
`POST /compile/prewarm`	none (free)	Warm Numba builtins / script IR
`WS /ws/run`	none (free)	AXIS WebSocket evaluate channel
`POST /preview/chart`	Pro usage	Chart thumbnail
`POST /preview/indicator`	Pro usage	Indicator-style preview
`POST /backtest/quick`	Pro usage	Equity / summary
`POST /auth/create_key`	admin token	Mint key
`GET /auth/usage`	Pro	Usage stats
`POST /auth/validate`	body key	Validate key
`POST /lsp/{completion,hover, diagnostics}`	none (free)	AXIS HTTP LSP bridge — see LSP
`/scripts` · `/cron/*`	`PYNE_RUNNER=1`	Hosted scripts + bar-close (runner)

Default local bind: `127.0.0.1:5002`.

Interface surface

Run the server (local)

```
pip install -r backend/requirements.txt

python -m backend.app
# or: make run
```

Health

```
curl -s http://127.0.0.1:5002/ | python -m json.tool
```

`POST /run`

Schema (`RUN_SCHEMA`):

Field	Type	Required	Default
`script`	string	yes	
`data`	list (OHLCV bars)	yes	
`symbol`	string	no	`"CHART"`
`data_source`	string	no	`""`
`data_options`	object	no	`{}`
`mode`	string	no	`"auto"`
`inputs`	object	no	`{}` — `input.*` by title (forces interpret under `auto`)
`libraries`	list	no	`[]` — `[{namespace, name, version, source}]` , max 32
`timeout_seconds`	number	no	omit — interpret wall-clock budget; passed only when set and `> 0`
`profiler`	bool	no	`false` — forces interpret
`webhook_url`	string	no	`""` (else env `ALERT_WEBHOOK_URL`)
`forward_alerts`	bool	no	`true`
`alert_last_bar`	bool	no	`true`
`alert_batch`	bool	no	`true`

Unknown extra keys → `UNKNOWN_FIELDS` . Query `?mode=` overrides only when the body omits `mode` .

Bar objects are expected to carry at least open/high/low/close/time (volume optional depending on script).

```
curl -s -X POST http://127.0.0.1:5002/run \
  -H 'Content-Type: application/json' \
  -d '{
    "script": "//@version=6\nindicator(\"Demo\")\nplot(ta.sma(close, 3))",
    "symbol": "AAPL",
    "mode": "interpret",
    "data": [
      {"time": 1, "open": 10, "high": 11, "low": 9, "close": 10.5, "volume": 1000},
      {"time": 2, "open": 10.5, "high": 12, "low": 10, "close": 11.5, "volume": 1100},
      {"time": 3, "open": 11.5, "high": 12.5, "low": 11, "close": 12, "volume": 1200}
    ]
  }'
```

Success shape (conceptual):

```
{
  "status": "success",
  "plots": [],
  "series": {},
  "plot_meta": {},
  "events": [],
  "drawings": [],
  "alerts": [],
  "script_id": "...",
  "run_id": "...",
  "count": 3,
  "mode": "interpret",
  "data_source": "chart"
}
```

When a webhook URL is configured, the response may also include `alert_forward` (delivery counts). See Alerts.

Error codes include `NO_SCRIPT`, `NO_DATA`, `DATA_SOURCE_ERROR`, `EXECUTION_ERROR`, plus schema codes `INVALID_BODY`, `MISSING_FIELD`, `INVALID_FIELD`, `UNKNOWN_FIELDS`.

`POST /run/batch`

Shared OHLCV, multiple scripts (max **8**). Envelope:

```
{
  "scripts": [
    {"id": "sma", "script": "//@version=6\nindicator(\"s\")\nplot(ta.sma(close, 5))"},
    {"id": "rsi", "script": "//@version=6\nindicator(\"r\")\nplot(ta.rsi(close, 14))"}
  ],
  "data": [/* bars */],
  "symbol": "AAPL",
  "mode": "interpret",
  "libraries": []
}
```

String entries in `scripts` are accepted and auto-id'd as `script_0`, ... Per-script errors do not necessarily fail the whole HTTP status (route returns 200 with per-item status — verify against deployment).

`POST /preview/chart` (Pro)

```
{
  "script": "",
  "data": {"close": [100, 101, 102], "volume": [1, 2, 3]},
  "options": {
    "type": "line",
    "color": "#2196F3",
    "width": 600,
    "height": 300,
    "show_volume": false
  }
}
```

Response includes base64 PNG `chart` + `meta`. Width/height clamped (e.g. max 1200×600).

`POST /preview/indicator` (Pro)

```
{
  "expression": "ta.sma(close, 20)",
  "data": {"close": [/* ... */]},
  "options": {}
}
```

`POST /backtest/quick` (Pro)

```
{
  "script": "//@version=6\nstrategy(\"x\")\n...",
  "data": {},
  "initial_capital": 10000.0,
  "mock_data": true,
  "mock_bars": 252
}
```

`POST /compile/prewarm` (free)

Warm Numba builtins / optional script list so the first interactive `/run` with `mode=auto|compile` skips cold JIT. Soft-fails without Numba (object-mode compile still works).

```
curl -s -X POST http://127.0.0.1:5002/compile/prewarm \
  -H 'Content-Type: application/json' \
  -d '{"force": true}' | python -m json.tool
```

Optional body: `scripts` (list of Pine sources to compile-warm), `force` (re-run builtins). Response includes timing / prewarm stats. Deploy hosts may also set `PYNE_COMPILE_PREWARM` for once-per-worker start. See Compiler overview.

Auth helpers

```
# Mint key (admin)
curl -s -X POST http://127.0.0.1:5002/auth/create_key \
  -H "Content-Type: application/json" \
  -H "X-Admin-Token: $ADMIN_TOKEN" \
  -d '{"tier":"hobby"}'

# Validate
curl -s -X POST http://127.0.0.1:5002/auth/validate \
  -H "Content-Type: application/json" \
  -d '{"api_key":"pyn_..."}'
```

Send Pro requests with the key as required by middleware (`Authorization: Bearer …` / documented API-key header — see Auth and keys).

There is **no** `pynescript.api.PynescriptAPI` client in this package. Call the HTTP contract with `requests` / `curl` as above.

Internals (repo paths)

Path	Role
`backend/app.py`	Routes `/`, `/run`, `/run/batch`, `/compile/prewarm`, auth
`backend/api/preview.py`	`/preview/*`, `/backtest/quick`
`backend/middleware/schemas.py`	Strict request schemas (`mode` default `auto`, alert flags)
`backend/middleware/auth.py`	Keys, usage, admin token
`src/pynescript/runtime/host.py`	Package Runtime SoT (`interpret` / `compile` / `auto`)
`backend/runtime.py`	Compat shim → `pynescript.runtime`
`backend/alert_forwarder.py`	L2 webhook POST batching
`backend/services/chart_renderer.py`	PNG rendering
`backend/services/backtest.py`	Quick backtest + mock OHLCV

Invariants & edge cases

- **Max body 5 MiB** — oversized JSON → Flask 413.
- **Strict schemas** — unknown fields rejected.
- **CORS** — browser clients need listed origins; server-to-server without `Origin` is fine.
- **`mode` default `auto`** — prefer warm compile; fall back to interpret (`auto_backend`, optional `compile_fallback_reason`). Strict `compile` errors instead of falling back; use `interpret` for alerts, `input.*` overrides, libraries, and full `request.*`.
- **`libraries` (0.3.7+)** — `[{namespace, name, version, source}]` binds `import ns/Name/ver` (max 32). `register_library_source` finalizes exports in 0.3.8; auto-mode keeps the list on interpret fallback. Same field on Flask `/run`, `/run/batch`, pyne-worker `/run`, and deployed scripts.
- **`timeout_seconds`** — optional interpret wall-clock budget on `/run` and `/run/batch`. Omit / `null` / `≤ 0` → no timeout. Exceeded runs surface `timed_out` on the HTTP body.
- **Alerts** — `alerts[]` on success (interpret). L2 webhooks: body `webhook_url` or env `ALERT_WEBHOOK_URL`; defaults last-bar + batch POST. See Alerts.
- **Batch cap 8** — `TOO_MANY_SCRIPTS` if exceeded.
- **Free-tier guards (0.3.4+; opt-in)** — process-local limits on unauthenticated `/run`, `/run/batch`, `/compile/prewarm` (and WS run). **Off unless** `FREE_TIER_LIMITS` is `1` / `true` / `yes` / `on`. Production compose sets that to `1`.

Env	Default	Role
`FREE_TIER_LIMITS`	off	Master switch for the rows below
`FREE_MAX_BARS`	5000	Max OHLCV bars (when on)
`FREE_MAX_SCRIPT_CHARS`	256 KiB	Max Pine source length (when on)
`FREE_MAX_CONCURRENT`	4	Simultaneous free runs per worker (when on)
`FREE_RATE_LIMIT` / `FREE_RATE_WINDOW_SEC`	60 / 60s	Sliding-window IP rate limit (when on)
Free `data_source`	`chart` / `mock` / `none` only	No outbound ccxt/yahoo on free paths (when on)
Numeric knobs treat `0` as unlimited. Health `features.free_tier_limits` reports the master switch. Webhook URLs stay SSRF-filtered regardless. Put a reverse proxy in front for multi-worker production.		

- AXIS multi-indicator UIs prefer `/run/batch` to share bar payloads; read `series` + `plot_meta` (including fill band refs) for charts.

Worked examples

Python `requests` client for `/run`

```
SCRIPT = """
//@version=6
indicator("SMA", overlay=true)
plot(ta.sma(close, 5))
"""

bars = [
    {"time": i, "open": 100 + i, "high": 101 + i, "low": 99 + i, "close": 100.5 + i, "volume": 1000
    for i in range(30)
]

r = requests.post(
    "http://127.0.0.1:5002/run",
    json={"script": SCRIPT, "data": bars, "symbol": "DEMO", "mode": "auto"},
    timeout=60,
)
r.raise_for_status()
payload = r.json()
assert payload["status"] == "success", payload
print(payload["count"], payload.get("mode"), list(payload.get("series", {})))
# optional: payload.get("alerts"), payload.get("alert_forward")
```

Batch two indicators

```
r = requests.post(
    "http://127.0.0.1:5002/run/batch",
    json={
        "scripts": [
            {"id": "sma", "script": "//@version=6\nindicator(\"s\")\nplot(ta.sma(close, 5))"},
            {"id": "ema", "script": "//@version=6\nindicator(\"e\")\nplot(ta.ema(close, 5))"},
        ],
        "data": bars,
    },
    timeout=120,
)
print(r.json())
```

Bind published libraries

```
r = requests.post(
    "http://127.0.0.1:5002/run",
    json={
        "script": '//@version=6\nimport Demo/Lib/1 as L\nindicator("t")\nplot(L.x)\n',
        "data": bars,
        "mode": "interpret",
        "libraries": [
            {
                "namespace": "Demo",
                "name": "Lib",
                "version": 1,
                "source": '//@version=6\nlibrary("Lib")\nexport x = 1.0\n',
            }
        ],
    },
    timeout=60,
)
```

Wire optional CCXT data source

```
{
  "script": "...",
  "data": [/* chart bars */],
  "symbol": "BTC/USDT",
  "data_source": "ccxt",
  "data_options": {"exchange": "binance"}
}
```

Requires server environment with `ccxt` installed.

Failure modes

Code / HTTP	Meaning	Action
400 `MISSING_FIELD`	No `script` / `data`	Fix body
400 `UNKNOWN_FIELDS`	Extra keys	Strip undocumented fields
400 `DATA_SOURCE_ERROR`	Provider config	Fix `data_options` / deps
500 `EXECUTION_ERROR`	Runtime exception	Simplify script; check message
403 on create_key	`ADMIN_TOKEN` unset/wrong	Export token
413	Body too large	Fewer bars / compress strategy
CORS browser error	Origin not allowed	Update `ALLOWED_ORIGINS`
Empty series	Script never plotted	Add `plot` / check declaration

See also

- Evaluate scripts
- Runtime modes
- Configuration
- API contract
- Auth and keys
- AXIS — AXIS over `/run` results
- HOOX — edge mesh after strategy events

TROUBLESHOOTING

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

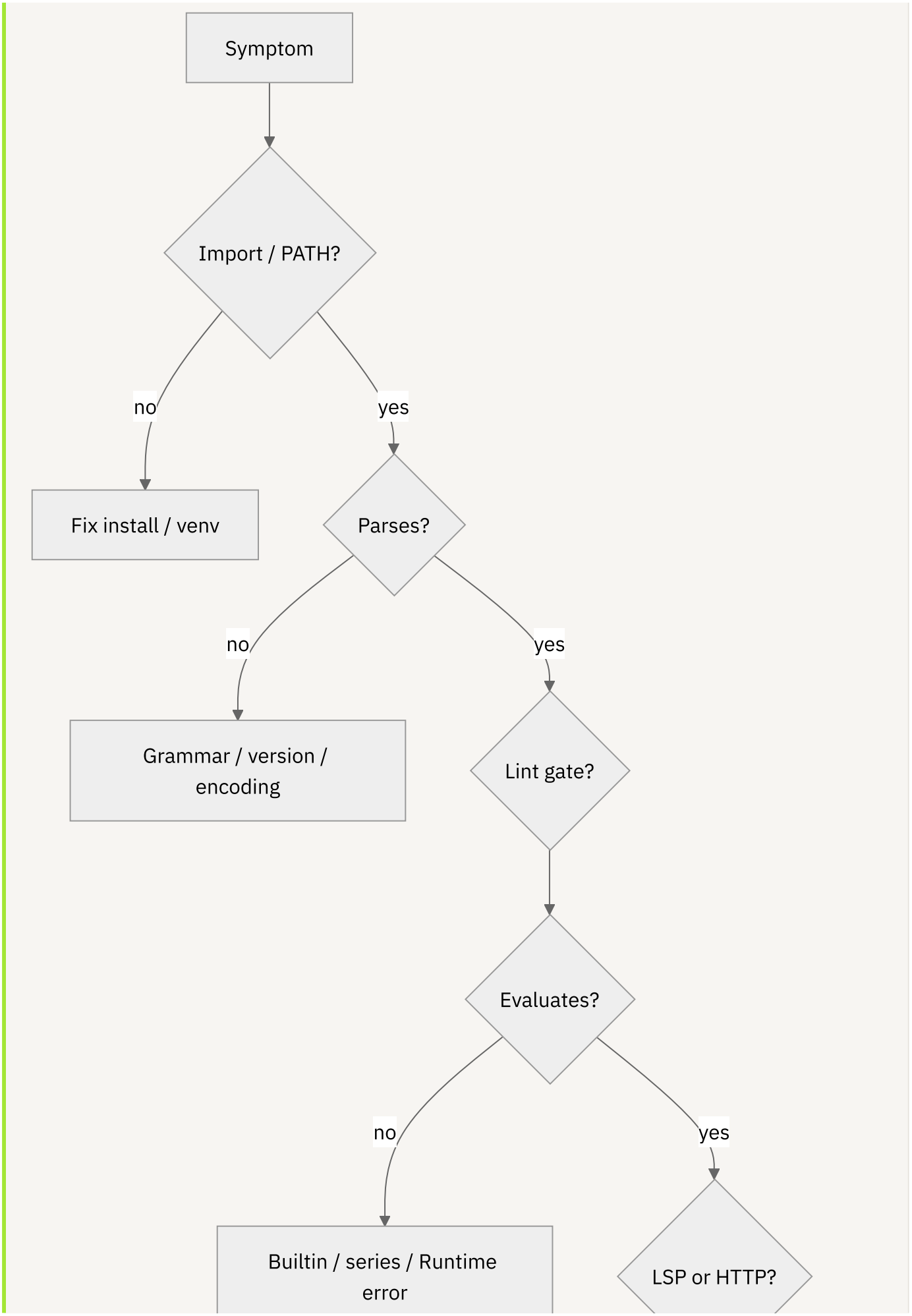
title: "Troubleshooting" description: "Diagnose pynescript install, parse, lint, evaluation, LSP, data providers, and Pro API failures with concrete checks."

TROUBLESHOOTING

Abstract

Failures cluster by layer: **environment**, **parse**, **lint**, **evaluation**, **data I/O**, **LSP**, **HTTP**. Work top-down: confirm the binary/module, then a minimal script, then full strategy complexity. This page is a runbook; deep protocol and runtime docs are linked per section.

Conceptual model


Symptom
Import / PATH?
no
yes
Fix install / venv
Parses?
no
yes
Grammar / version / encoding
Lint gate?
Evaluates?
no
yes
Builtin / series / Runtime error
LSP or HTTP?



Interface surface (diagnostic commands)

```
python -c "import pynescript; from pynescript import __about__; print(__about__.__version__)"
python -c "from pynescript.ast import parse, unparse; print(unparse(parse('//@version=6\nindicator(
pyne --version
pyne lint - <<< $'//@version=6\nindicator("t")\nplot(close)\n'
python -c "import pygls"  # LSP extra
python -c "import ccxt"   # data extra
curl -s http://127.0.0.1:5002/ | head
```

Internals (where errors originate)

Layer	Path
CLI	`src/pynescript/__main__.py`
Parse	`src/pynescript/ast/helper.py` , ANTLR error listener
Lint	`src/pynescript/ast/linter.py`
Eval	`src/pynescript/ast/evaluator/` , `src/pynescript/runtime/`
Data	`src/pynescript/util/data.py`
LSP	`src/pynescript/langserver/`
HTTP	`backend/app.py` , `backend/middleware/schemas.py`

Invariants & edge cases

- Minimal green script for isolation:

```
//@version=6
indicator("t")
plot(close)
```

- Encoding must be UTF-8 unless you pass `--encoding` .
- Pine v4 and earlier trigger linter deprecation (`W002`); parser support may still differ by construct.
- Recursion limit is raised only during parse; extreme generated scripts can still fail.

Worked examples (by symptom)

1. `pyne` / `pynescript: command not found`

```
which python
python -m pynescript --help
# If module works but script does not:
python -m pip show hoox-pyne
ls "$(python -c 'import sysconfig; print(sysconfig.get_path("scripts"))')"
```

Fix: activate the venv used for install; reinstall `pip install hoox-pyne` (import package is still `pynescript`). Do **not** `pip show pynescript` — that is a different PyPI project.

2. `ModuleNotFoundError: pynescript`

Wrong interpreter (IDE using system Python). Point the IDE at the venv; reinstall editable from repo root: `pip install -e .`.

3. Parse / SyntaxError

```
pyne parse-and-dump broken.pine
```

Checks:

- `//@version=6` present (v5 still accepted)
- Balanced parentheses / indentation (Pine uses line structure)
- Multiline strings / v6 features need a package version that includes them
- Binary/null bytes in file

Reduce to a minimal repro; if TV accepts it and PYNE does not, consult missing features and file a grammar issue.

4. Round-trip looks “wrong”

`unparse` normalizes formatting. Compare dumps:

```
pyne parse-and-dump a.pine > /tmp/a.ast
pyne parse-and-unparse a.pine | pyne parse-and-dump /dev/stdin
# stdin may not work for parse-and-dump (file path required) — use a temp file:
pyne parse-and-unparse a.pine --output-file /tmp/a2.pine
pyne parse-and-dump /tmp/a2.pine > /tmp/a2.ast
diff -u /tmp/a.ast /tmp/a2.ast
```

Structural diff empty ⇒ cosmetic only.

5. Lint noise / CI red

Code	Action
`E001`	Fix syntax first
`W001`	Add `//@version=6`
`W002`	Upgrade version pragma
`C001` – `C004`	Style; use `--fail-on errors` if style should not gate CI

```
pyne lint script.pine --fail-on errors
```

6. `literal_eval` fails

- Use `mode="eval"`-compatible **expressions**, not full `strategy()` scripts
- Pass series as Python lists inside the expression or via `context`
- Missing builtin → incomplete implementation

Escalate to `Runtime.run` for multi-bar scripts.

7. Runtime / `/run` execution error

```
curl -s -X POST http://127.0.0.1:5002/run -H 'Content-Type: application/json' -d '{"script":"//@ver
```

If minimal works but strategy fails: remove `request.*`, drawings, then strategy entries until isolated. Check `mode`; try `"interpret"`. Library `Runtime.run` defaults to **interpret**; `POST /run` defaults to **auto**; `pyne run` is compile-only synthetic smoke — they will not match each other unless you set `mode` explicitly. See modes.

8. Schema validation errors

`UNKNOWN_FIELDS` means you sent a key not in the schema — remove it. Do not rely on servers ignoring extras.

9. CORS from AXIS / browser

```

```

Restart Flask after changing env.

10. LSP will not start

```
pip install "hoox-pyne[lsp]"
python -c "import pygls, lsprotocol"
command -v pyne-lsp   # alias: pynescript-lsp
# Run with editor logging enabled; check stderr for import traces
```

Ensure the editor's `command` matches the venv. For Neovim, confirm `filetypes` include your buffer's filetype.

11. Data provider failures

Provider	Checklist
`mock`	Should always work offline
`yahoo`	Network; symbol format
`alphavantage`	API key; demo limits
`ccxt`	`pip install "hoox-pyne[data]"`; exchange id; symbol `BTC/USDT`

```
pyne data AAPL --provider mock
```

12. Version / parity confusion

PYNE is independent of TradingView®. Behavioral gaps are expected for edge builtins, broker emulators, and UI-only calls. Use compatibility.

Failure modes (quick matrix)

Symptom	Layer	First check
Command not found	env	`python -m pynescript`
Import error pygls	extra	`[lsp]`
Import error ccxt	extra	`[data]`
SyntaxError	parse	minimal script + version
Lint CI fail	lint	`--fail-on` level
Empty plots	eval	`plot()` present; enough bars
`pyne run` ≠ `Runtime.run`	modes	CLI is compile-only synthetic; library default is interpret
`UNKNOWN_FIELDS` + `timeout_seconds`	HTTP	field is valid on `/run` and `/run/batch` (0.3.11+); other extra keys still 400
400 UNKNOWN_FIELDS	HTTP	schema
403 create_key	HTTP	`ADMIN_TOKEN`
413	HTTP	body size
Squiggles missing	LSP	diagnostics enabled + filetype
Numerical drift	eval	warmup / parity docs

See also

- Installation
- Configuration
- CLI guide
- Evaluate scripts
- Pro API usage

- Editors
- FAQ

CLI COMMANDS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "CLI Commands Reference" description: "Complete option reference for the pyne Click CLI: check, format, lint, compile, run, optimize, prewarm, data, info, and more."

CLI COMMANDS REFERENCE

Abstract

Machine-oriented reference for every command on the preferred `pyne` Click group (alias `pynescript`; `src/pynescript/__main__.py`). Install via `hoox-pyne` (`pip install hoox-pyne`). For workflows see the CLI guide. The language server is a **separate** console script (`pyne-lsp`; alias `pynescript-lsp`).

Version option: `pyne --version` / `-V` reports `pynescript.__about__.__version__` (e.g. `0.6.7`). Global `--no-color` honors `NO_COLOR`.

Command index

Command	Aliases	Purpose
`check`		Parse-only validation (CI-friendly exit codes)
`format`	`fmt`	Parse → unparse (structural format)
`convert`		Rewrite older Pine toward v5 or v6
`parse-and-dump`	`dump`, `ast`	AST dump
`parse-and-unparse`	`unparse`	Round-trip source
`lint`		Static diagnostics (JSON / fail-on)
`compile`		Transpile or compile-check (Numba path)
`prewarm`		Warm Numba builtins / script IR caches (H2)
`run`		Compile + execute on synthetic OHLCV
`optimize`		Search `input.*` values over N interpret Runtime runs
`data`		Fetch market bars
`runner`		Hosted scripts on the Pro API (`deploy` / `tick` / `list`)
`info`	`ls`	Version + optional extras

```
pyne --help
pyne --version
pyne <command> -h
# alias: pynescript …
```

`pyne check`

Validate that path(s) parse as Pine Script.

Kind	Name	Default	Description
argument	`PATHS…`	(optional)	Files and/or directories; omit or `-` → stdin
option	`--encoding`	`utf-8`	Text encoding
option	`-q, --quiet`	off	Exit code only
option	`--ext`	`.pine`	When PATH is a directory, only this suffix

```
pyne check script.pine
pyne check scripts/ --ext .pyne
pyne check -q strategy.pine   # exit 0/1 only
cat x.pine | pyne check -
```

Exit: `0` all ok; `1` any parse failure.

`pyne format` (`fmt`)

Canonicalize Pine by parse → unparse (structural, not a full style formatter).

Kind	Name	Default	Description
argument	`PATH`	required	File or `-` (stdin)
option	`--encoding`	`utf-8`	
option	`-w, --write`	off	Write back to PATH (not for stdin)
option	`--check`	off	Exit `1` if format would change (no write)
option	`-o, --output-file`	`-`	Output when not using `--write`

```
pyne format script.pine            # stdout
pyne fmt script.pine -w
pyne format script.pine --check    # CI drift gate
```

`pyne convert`

Rewrite older Pine toward v5 or v6. `--to 6` converts **any** older version (missing `//@version` counts as v1): color / `n` / timeframe idents, `ta.*` / `math.*` / `request.*` namespaces, `study(` → `indicator(`, typed `input.*()`, `iff` / `offset`. Not a semantic v6 migrator (bool-as-int, `na`).

Kind	Name	Default	Description
argument	`PATH`	required	File or `-` (stdin)
option	`--to`	required	`5` / `6` / `v5` / `v6`
option	`--encoding`	`utf-8`	
option	`-w, --write`	off	Write back to PATH (not for stdin)
option	`-o, --output-file`	`-`	Output when not using `--write`

```
pyne convert old.pine --to 6
pyne convert strategy.pine --to 6 -w
```

`pyne parse-and-dump` (`dump` / `ast`)

Kind	Name	Default	Description
argument	`PATH`	required	Input file
option	`--encoding`	`utf-8`	
option	`--indent`	`2`	Dump indent
option	`-o, --output-file`	`-`	Output path

```
pyne dump strategy.pine
pyne parse-and-dump strategy.pine --indent 4 -o tree.txt
```

`pyne parse-and-unparse` (`unparse`)

Kind	Name	Default	Description
argument	`PATH`	required	Input file
option	`--encoding`	`utf-8`	
option	`-o, --output-file`	`-`	Output path

```
pyne unparse messy.pine -o clean.pine
```

`pyne lint`

Kind	Name	Default	Description
argument	`PATH`	optional	File; omit or `-` → stdin
option	`--encoding`	`utf-8`	
option	`--fail-on`	`errors`	`errors` | `warnings` | `all` | `never`
option	`--json`	off	JSON array of findings
option	`-q, --quiet`	off	Summary only

```
pyne lint strategy.pine
pyne lint --json --fail-on never strategy.pine
pyne lint --fail-on warnings strategy.pine
cat strategy.pine | pyne lint -
```

`pyne compile`

Compile via `pynescript.compiler.compile_script`. Requires `pip install "hoox-pyne[compile]"` for full Numba path; `--emit` works without JIT.

Kind	Name	Default	Description
argument	`PATH`	required	Pine file
option	`--encoding`	`utf-8`	
option	`--emit`	off	Print generated Python only (no exec/JIT)
option	`-o, --output-file`	`-`	Where to write `--emit` output
option	`--time / --no-time`	time on	Print compile timing

```
pyne compile script.pine --emit
pyne compile script.pine --emit -o out.py
pyne compile script.pine            # warm load / compile-check
```

`pyne prewarm`

H2 warm-compile product path: pay cold JIT before first interactive run.

Kind	Name	Default	Description
argument	`PATH…`	optional	Pine files to compile into IR caches
option	`--encoding`	`utf-8`	
option	`--force`	off	Re-run builtin warm-up
option	`--json`	off	Machine-readable summary

```
pyne prewarm                    # shared Numba kernels only
pyne prewarm strategy.pine lib/*.pine
pyne prewarm --json
```

`pyne run`

Compile and execute on deterministic synthetic OHLCV (same compile pipeline as Pro API `mode=compile`). **No `--mode`.** Does **not** call `Runtime.run`. Prefer package `pynescript.runtime.Runtime` / Pro API for real bars, `libraries=`, and `timeout_seconds`.

Kind	Name	Default	Description
argument	`PATH`	required	Pine file
option	`--encoding`	`utf-8`	
option	`--bars`	`50`	Synthetic bar count
option	`--json`	off	Plot series summary as JSON
option	`-q, --quiet`	off	Errors / exit only

```
pyne run strategy.pine --bars 100
pyne run strategy.pine --json
```

`pyne optimize`

Search `input.*` values for a **`strategy()`** script. Loops interpret `Runtime` (not `/backtest/quick`). Requires `--space` JSON.

Kind	Name	Default	Description
argument	`PATH`	required	Pine file (`-` = stdin)
option	`--encoding`	`utf-8`	
option	`--bars`	`120`	Synthetic bar count (min 8)
option	`--trials`	`20`	Trial budget (max 200)
option	`--space`	required	JSON `{"params":[{"name","kind","min","max"}]}`
option	`--sampler`	`auto`	`auto` | `random` | `tpe` | `grid`
option	`--objective`	`composite`	`composite` | `net_pnl` | `profit_factor` | `calmar`
option	`--validation`	`holdout`	`holdout` | `walk-forward` | `in-sample`
option	`--train-bars`	derive	Walk-forward train window
option	`--test-bars`	derive	Walk-forward test window
option	`--step-bars`	derive	Walk-forward step (default: the test window)
option	`--holdout-frac`	`0.3`	Holdout test fraction
option	`--seed`	unset	RNG seed
option	`--json`	off	Full study JSON

`--validation walk-forward` derives windows from `--bars` when the bar flags are omitted: `train=max(8, bars*2//3)`, `test=max(4, bars//6)`, `step=max(1, test)` (so default `--bars 120` → 80/20/20). Explicit `--train-bars` / `--test-bars` / `--step-bars` override that. The CLI refuses walk-forward when `--bars < train+test`. All-errored studies exit 1 even if the study status is `success`.

```
pyne optimize strat.pine --trials 20 --sampler tpe \
  --space '{"params":[{"name":"Fast","kind":"int","min":3,"max":12}]}'
pyne optimize strat.pine --validation walk-forward --bars 120 --trials 20 \
  --space '{"params":[{"name":"Fast","kind":"int","min":3,"max":12}]}'
pyne optimize strat.pine --validation walk-forward --train-bars 80 --test-bars 20 \
  --space '{"params":[{"name":"Fast","kind":"int","min":3,"max":12}]}'
```

Library: `pynescript.optimize.run_study`. HTTP: POST /optimize.

`pyne data`

Kind	Name	Default	Description
argument	`SYMBOL`	required	Ticker / market symbol
option	`--provider`	`mock`	`mock` | `yahoo` | `alphavantage` | `ccxt`
option	`--period`	`1y`	e.g. `6mo`, `1y`
option	`--interval`	`1d`	e.g. `1h`, `1d`
option	`--api-key` / `--secret`	empty	Provider credentials
option	`--exchange`	`binance`	CCXT exchange id
option	`--format`	`table`	`table` | `json` | `csv`
option	`-o, --output-file`	`-`	Output path

```
pyne data AAPL
pyne data AAPL --provider yahoo --period 6mo
pyne data BTC/USDT --provider ccxt --exchange binance --format csv
```

CCXT needs `pip install "hoox-pyne[data]"`.

`pyne runner`

HTTP client for the optional Flask hosted runner (`PYNE_RUNNER=1`). Group options apply to every subcommand.

Kind	Name	Default	Description
option	`--url`	`http://127.0.0.1:5002`	Pro API origin (`PYNE_API_URL`)
option	`--token`	empty	`X-Admin-Token` (`ADMIN_TOKEN`)

Subcommand	Role
`deploy PATH`	Read a `.pine` / `.pyne` file and `POST /scripts`
`list`	`GET /scripts`
`show ID`	`GET /scripts/:id`
`delete ID`	`DELETE /scripts/:id`
`tick`	`POST /cron/run` (`--id`, `--force`)
`jobs`	`GET /cron/jobs`
`enable ID` / `disable ID`	`PUT /cron/jobs`

```
pyne runner deploy script.pine --id demo --symbol BTCUSDT --source mock
pyne runner tick --id demo --force
pyne runner list
pyne runner --url http://127.0.0.1:5002 jobs
```

Full HTTP contract: hosted runner.

`pyne info` (`ls`)

Kind	Name	Description
option	`--json`	Machine-readable payload

Prints package version, Python, platform, and whether `numba` / `rich` are available.

Related: `pyne-lsp` (not a subcommand)

```
pip install "hoox-pyne[lsp]"
pyne-lsp
# alias: pynescript-lsp
python -m pynescript.langserver
```

Internals

Item	Path
CLI group	`src/pynescript/__main__.py`
Entry	`pyproject.toml` → `pyne` / `pynescript` = `pynescript.__main__:cli`
parse/dump/unparse	`src/pynescript/ast/helper.py`
lint	`src/pynescript/ast/linter.py`
compile / prewarm	`src/pynescript/compiler/`
optimize	`src/pynescript/optimize/`
data	`src/pynescript/util/data.py`

Invariants

- Aliases: `dump` / `ast` → `parse-and-dump`; `unparse` → `parse-and-unparse`; `fmt` → `format`; `ls` → `info`.
- Prefix match works when unique (`pynescript ch` → `check`).
- `check` walks directories; dump/format take a single file.
- **Docker:** `pynescript-cli` image `ENTRYPOINT` is `pynescript` (non-root).
- **Standalone binaries:** GitHub Release `pynescript-cli-*` (Nuitka).

Failure modes

Situation	Result
Parse failure (`check` / dump)	Non-zero exit
Format `--check` drift	Exit `1`
Lint threshold	Non-zero per `--fail-on`
Compile without numba (full path)	Clear error; `--emit` still works
Provider failure	Click error with message
Missing `ccxt` for data	Import/provider error → install `[data]`

See also

- CLI guide
- Runtime modes
- Installation
- Compiler overview
- Docker

MODES

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Runtime modes" description: "interpret / compile / auto defaults differ by surface. Library interpret, POST /run auto, pyne run compile-only. PyneTS 0.5.0 has interpret + JS compile."

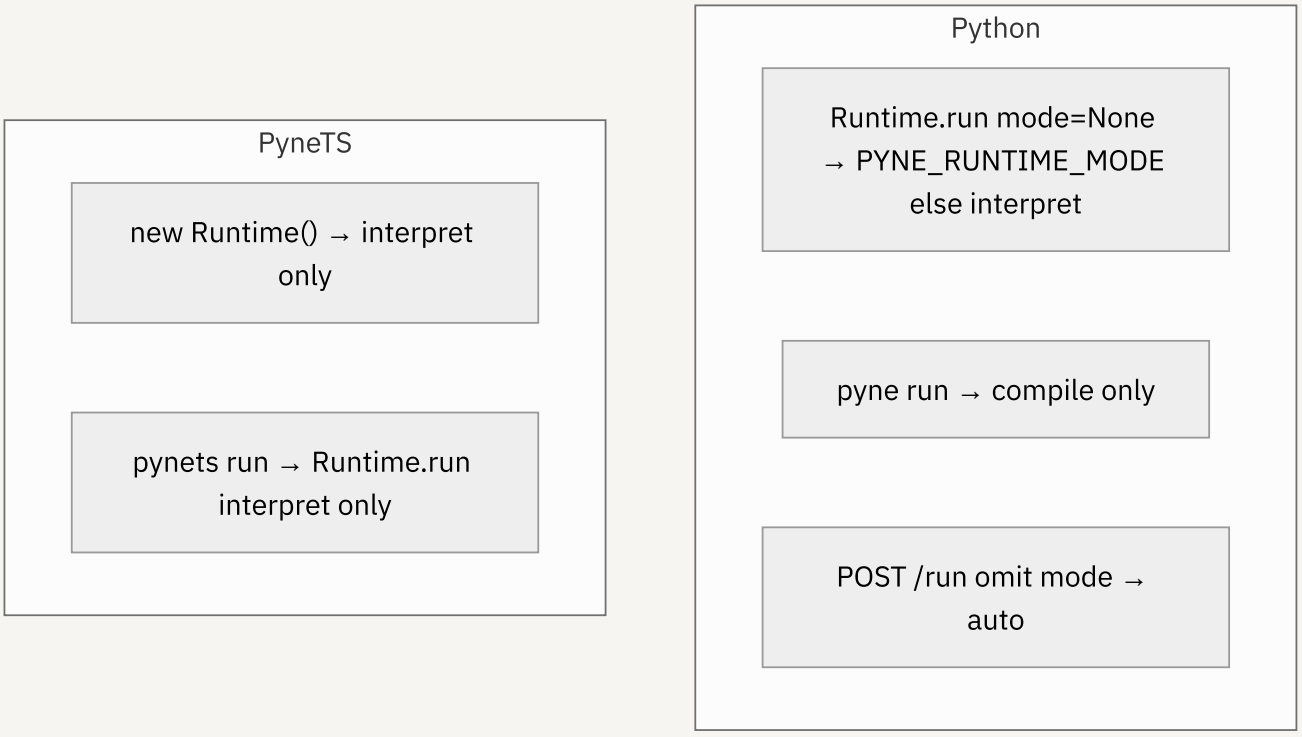
RUNTIME MODES

Abstract

`mode` selects how the bar loop executes: **`interpret`** (AST walk), **`compile`** (transpiled loop), or **`auto`** (try compile, fall back to interpret). The **default is not the same on every surface**. This is the most common footgun when moving a script from the library to the Pro API or between Python and PyneTS.

There is **no** `Runtime.evaluate`.

Conceptual model



Interface surface

What each mode means

Mode	Python	PyneTS (this checkout)
`interpret`	AST walker. Full host: `input.*`, `request.*`, libraries, profiler, alerts	AST walker only (`pynets/src/runtime/interpret.ts`). Result `mode` is always `"interpret"`
`compile`	Numba nopython **or** object-mode Python loop	**Not implemented** — no `src/runtime/compile/` in this tree
`auto`	Try compile; on ineligibility / compile error / compiled runtime error → interpret	**Not implemented**

Defaults by surface

Surface	Default `mode`	Calls `Runtime.run`?
`pynescript.runtime.Runtime.run(..., mode=None)`	`PYNE_RUNTIME_MODE` else **`interpret`**	yes
Pro API `POST /run` body omits `mode`	schema **`auto`**	yes (host wrap)
`?mode=` query (legacy)	overrides body	yes
CLI `pyne run`	**compile only** — not `Runtime.run`	**no**
`new Runtime()` / `pynets run`	**interpret only** (no `--mode`; `--bars` default **20**)	yes
AXIS engine `server`	whatever you pass; UI default often `auto`	HTTP `/run`
AXIS engine `pyodide`	object-mode in Wasm (no Numba)	in-browser Python

`Runtime.run` (Python, exact)

```
Runtime(symbol="AAPL").run(
    source_code,
    ohlcv_data,
    data_feed=None,
    data_provider=None,
    mode=None,            # env PYNE_RUNTIME_MODE else "interpret"
    inputs=None,
    profiler=False,
    timeout_seconds=None,
    libraries=None,
    realtime_last_bar=False,
    realtime_ticks=1,
    realtime_bars=0,
    realtime_from_bar=None,
)
```

Argument	Surfaces	Effect on `mode`
`mode`	library / `POST /run`	See defaults table. `pyne run` has no `--mode`.
`inputs`	library / `POST /run`	Non-empty overrides **force interpret** (compile does not apply them).
`libraries`	library / `POST /run` / `POST /run/batch` (max 32)	`[{namespace, name, version, source}]` for `import ns/Name/ver`. Compile-ineligible; `auto` keeps the list on interpret fallback. **Not** on `pyne run`.
`timeout_seconds`	library / `POST /run` / `POST /run/batch`	Interpret wall-clock budget. Exceeded → partial plots + `timed_out`. Omit / `null` / `≤ 0` → no timeout. **Not** on `pyne run`.
`profiler`	library / `POST /run`	Forces interpret (line timings).
`realtime_*`	library only	Interpret forming-bar window (`realtime_last_bar`, `realtime_ticks`, `realtime_bars`, `realtime_from_bar`).

`inputs` / `profiler` / `libraries` / `timeout_seconds` / `realtime_*` force or keep the **interpret** host (or force interpret after compile miss).

`Runtime.run` (PyneTS)

```
new Runtime(symbol, { inputs, broker, timeframe })
  .run(source, ohlcv, extra?)
```

There is **no** constructor/`extra.mode` and **no** `libraries` option on this host. `RuntimeResult.mode` is always `"interpret"`.

Internals

Surface	Implementation
Python library	`src/pynescript/runtime/host.py`
Python CLI `run`	`src/pynescript/__main__.py` → `compile_script` + synthetic bars
Pro API	`backend/app.py` `RUN_SCHEMA` default `auto`
PyneTS	`@hoox-sh/pynets` **0.5.0** — interpret + JS compile (`mode` `interpret` / `compile` / `auto`). Python remains the oracle
PyneTS CLI	`pynets/src/cli.ts` → `new Runtime("AAPL", { broker }).run`

Invariants & edge cases

1. **Do not document `pyne run` as `mode=auto`.** It is compile-only smoke (`--bars` default 50).
2. **Do not document `pynets run` as Python `pyne run`.** PyneTS `Runtime.run` defaults to interpret; **0.5.0** also has JS `compile` / `auto`. Default `--bars` is **20** (Python `pyne run` defaults to **50**).
3. `result.mode` is what **executed**. After `auto`, also read `auto_backend` and `compile_fallback_reason`.
4. Foreign `request.security` is `na` on every host. Compile does not invent bars to become "more complete".
5. AXIS Pyodide `compile` / `auto` cannot be Numba (Wasm).

Worked examples

Force interpret everywhere you care about `input.*` + libraries + alerts:

```
Runtime(symbol="AAPL").run(src, bars, mode="interpret", inputs={"len": 14})
```

```
new Runtime("AAPL", { inputs: { len: 14 } }).run(src, bars);
```

```
# Python desk smoke (compile only)
pyne run script.pine --bars 50

# PyneTS desk smoke (Runtime.run; default interpret, --bars 20)
pynets run script.pine --bars 50
```

Failure modes

Symptom	Cause	Fix
Pro API "misses" an input	`auto` compiled a path that drops overrides	Pass `mode: "interpret"`
`pyne run` ≠ notebook `Runtime.run`	Different host	Use the library or Pro API
Expect PyneTS ≡ Python compile	JS emit is object-mode analog, not Numba	Treat Python Runtime as the oracle
`mode` omitted, two hosts disagree	Different defaults	Set `mode` explicitly

See also

- Evaluate scripts
- Python compiler
- PyneTS compile
- Configuration
- Evaluate contract

GLOSSARY

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Glossary" description: "Definitions of PYNE end-user terms: AST, ASDL, bar-loop, series, Runtime, LSP, Pro API, extras, and related concepts."

GLOSSARY

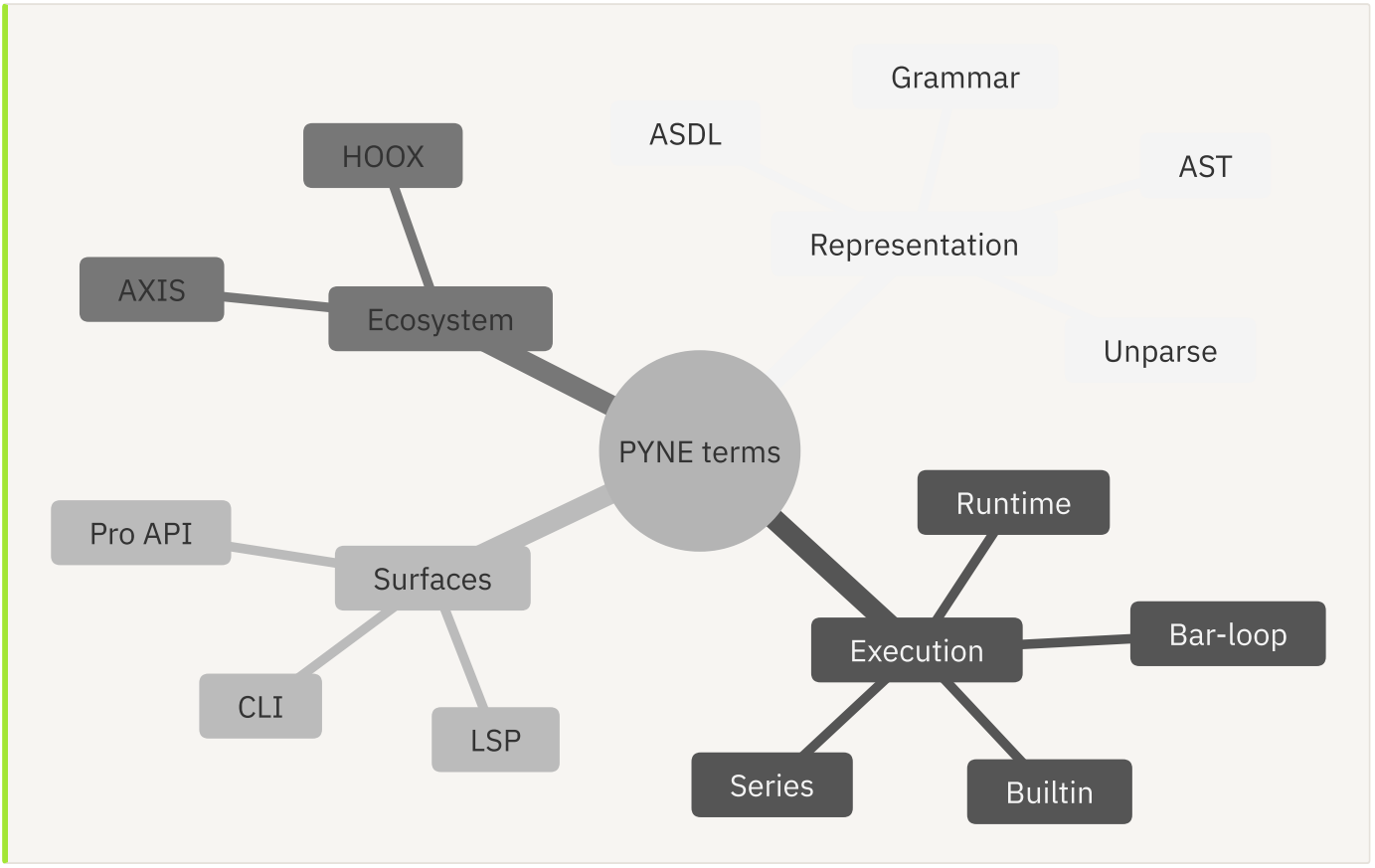
Abstract

Shared vocabulary for the End User track. Terms are defined as **used in this repo**, not as MarketingView marketing copy. Cross-links point at deeper systems manuals where useful.

Conceptual model

Terms fall into four buckets:

1. **Language representation** — grammar, AST, unparser
2. **Execution** — series, bar-loop, builtins, strategy events
3. **Surfaces** — CLI, library, LSP, Pro API
4. **Ecosystem** — AXIS, HOOX, PyneTS, pyne-worker, pyne-agent-worker



Interface surface

This page is pure reference — no commands required.

Internals (repo paths)

Term cluster	Primary paths
Parse / AST	`src/pynescript/ast/helper.py`, `builder.py`, `grammar/`
Evaluate	`src/pynescript/ast/evaluator/`, `src/pynescript/runtime/` (`backend/runtime.py` is a compat shim)
Lint	`src/pynescript/ast/linter.py`
LSP	`src/pynescript/langserver/`
HTTP	`backend/app.py`

Invariants & edge cases

- **Pine Script™ / TradingView®** are trademarks of TradingView, Inc. PYNE is independent (see product index).
- “Parity” is empirical (tests/corpus), not a legal claim of equivalence.

Terms

ASDL

Abstract Syntax Description Language. Schema language used to generate Python AST node classes (`Pinescript.asdl` → generated node module). Gives a single algebraic description of script structure.

Annotation

Special comments such as `//@version=6` (preferred) attached to script or declaration nodes during parse (`_add_annotations` in `helper.py`).

AST

Abstract Syntax Tree. In-memory tree of ASDL-generated nodes produced by `parse`. Root is typically a `Script` (`mode="exec"`) or `Expression` (`mode="eval"`).

AXIS

Optional **charting PWA "AXIS"** product (`frontend/`, docs at `/axis/docs`). Consumes evaluate results (series/plots); not required for evaluation.

Bar / bar-loop

A discrete OHLCV time step. A **bar-loop** evaluator re-executes script logic once per bar with updated series history (`pynescript.runtime.Runtime.run`).

Builtin

Host-provided namespace function or value (`ta.rsi`, `strategy.entry`, `math.sqrt`, ...). Implemented under `ast/evaluator/builtins/` and catalogued for LSP metadata.

CLI

The preferred **`pyne`** Click command group (alias `pynescript`: `check`, `format`, `lint`, `compile`, `prewarm`, `run`, `data`, `info`). `pyne run` is compile-only synthetic smoke — not `Runtime.run`.

Compile mode

`mode="compile"` on Runtime / `/run`: attempts a **Numba (or similar) compiled subset** of bar logic for speed. Not full language coverage.

Data provider

Historical OHLCV source (`mock`, `yahoo`, `alphavantage`, `ccxt`) via `pynescript.util.data`.

Data feed

Realtime stream abstraction (`datafeed`, e.g. CCXT Pro) for live candles/trades.

Dump

`dump(node)` — pretty textual representation of an AST for debugging and tests (not executable Pine).

Evaluate / evaluator

Execution of AST nodes to values. Ranges from `literal_eval` (expression-safe) to full statement/builtin evaluators and package **`pynescript.runtime.Runtime`** (Pro API / workers share the same host).

Extra (package extra)

Optional dependency set in `pyproject.toml`: `lsp`, `dev-lsp`, `compile`, `data`, `datafeed`, `pro`.

HOOX

Edge **trade execution mesh** (separate monorepo / docs under `/docs`). Can consume strategy events from evaluate pipelines; optional.

Interpret mode

Default Runtime mode: AST walking per bar without the compiled subset path.

`libraries=`

Optional `Runtime.run` / `POST /run` list of `{namespace, name, version, source}` dicts registered before `import ns/Name/ver` resolves. Compile-ineligible. HTTP cap is 32. Not available on `pyne run`.

`literal_eval`

Helper that parses an expression (or accepts an AST) and evaluates literals plus supported builtins/series context — analogous in spirit to Python's `ast.literal_eval` but Pine-aware and broader.

Lint / `LintWarning`

Static diagnostics with `code`, `message`, `line`, `severity` (`error` | `warning`). Entry: `lint_script`.

LSP

Language Server Protocol. `pyne-lsp` process (alias `pynescript-lsp`) implementing diagnostics, completion, hover, navigation, formatting.

`once`

August 2026 Pine **conditional structure** (`hoox-pyne` **0.4.4**). `once [cond]` runs its indented block the first time the condition is true on a **closed** bar, then never again. Optional condition defaults to `true`. Unlike `if` / `switch`, it **does not return a value** (`x = once …` does not parse). Interpret may re-run the body on unconfirmed realtime ticks until `barstate.isconfirmed`. See Expressions & statements.

NodeVisitor / NodeTransformer

Patterns for traversing or rewriting ASTs (`visitor.py`, `transformer.py`).

OHLCV

Open, high, low, close, volume — bar fields passed to Runtime / `/run` (plus `time`).

PyneTS

TypeScript / Bun library (`@hoox-sh/pynets`). Public names match Python: `parse`, `unparse`, `Runtime.run`. **Not** a Worker and **not** pyne-worker. See PyneTS.

pyne-agent-worker

Sister Cloudflare Worker that turns natural language into Pine source (`POST /v1/chat`). Standalone by default; optional validate via pyne-worker. See pyne-agent-worker.

pyne-worker

Python Cloudflare Worker (sister repo) embedding pynescript for edge `POST /run` . See pyne-worker. HOOX mesh notes: isolate profile.

Plot / series / plot_meta

Evaluate outputs: plotted values, named multi-series maps, and metadata for AXIS UIs (AXIS).

Pro API

Flask application in `backend/` exposing `/run` , preview, backtest, and auth endpoints.

PYNE

Product name for this evaluation stack (docs under `/pyne/docs`). PyPI distribution is **`hoox-pyne`** ; the import package is **`pynescript`** .

Round-trip

`parse` → `unparse` (optionally → `parse` again). Used to normalize formatting and test fidelity.

Runtime

`pynescript.runtime.Runtime` — multi-bar execution engine (`src/pynescript/runtime/host.py`). `backend.runtime` re-exports the same class for the Pro API. Library default `mode` is **interpret**; `POST /run` defaults to **auto**. See modes.

Series

Time-indexed value stream with history access (`close[1]`). Implementation includes `PineSeries` and evaluator series objects.

Strategy event

Structured record of entries/exits/orders emitted during strategy evaluation for downstream systems (HOOX, analytics).

`timeout_seconds`

Optional `Runtime.run` wall-clock budget (interpret path). Also a `POST /run` / `/run/batch` body field (0.3.11+). Exceeded runs return partial plots with `timed_out` . Omit / `null` / `≤ 0` → no timeout.

Optimize (hyperparameter search)

`pynescript.optimize.run_study` / `pyne optimize` / `POST /optimize` (0.3.12+). Loops interpret `Runtime` with `input.*` overrides for a `strategy()` script. Not a Pine builtin and not `/backtest/quick` .

`timeframe.change`

`timeframe.change(tf)` is true on the first bar of a new UTC fixed-width period (0.3.12+). Not exchange-calendar / DST aware. See known divergences.

Unparse

`unparse(node)` — regenerate Pine source text from an AST.

`//@version`

Version pragma annotation; linter expects v5+ for modern scripts.

Warmup

Bars required before a TA function has enough history; early outputs may be `na`.

Worked examples

Using terms together:

```
Source --parse--> AST --unparse--> normalized source
AST + OHLCV --Runtime bar-loop--> series + strategy events
AST --lint_script--> LintWarnings
AST --LSP--> editor diagnostics/completion
series --AXIS--> chart
events --HOOX--> exchange orders
```

Failure modes

Misusing terms often causes support confusion:

Confused pair	Distinction
dump vs unparse	dump is debug AST text; unparse is Pine source
literal_eval vs Runtime	single-shot expressions vs multi-bar scripts
pyne vs pyne-lsp	desk CLI vs language server (aliases: pynescript*)
AXIS vs PYNE	AXIS vs evaluate engine
data provider vs data feed	historical fetch vs realtime stream

See also

- End User hub
- FAQ
- PYNE product map
- AXIS docs
- HOOX docs

FAQ

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "FAQ" description: "Frequently asked questions about pynescript install, Pine version support, TradingView parity, CLI vs library, LSP, Pro API, licensing, and ecosystem boundaries."

FAQ

Abstract

Short answers to questions that recur when adopting PYNE as a **consumer**. Where behavior is evolving, answers point at status docs rather than overclaiming parity.

Conceptual model

Questions group into: install & package shape, language coverage, evaluation fidelity, tooling surfaces, and product boundaries (AXIS / HOOX / workers).

Interface surface

No dedicated FAQ API — pointers only.

Internals (repo paths)

Answers cite:

- `pyproject.toml` — version, extras, license
- `src/pynescript/__main__.py` — CLI reality
- `src/pynescript/runtime/host.py` — `Runtime.run`
- `docs/missing_features.md` / reference status pages — coverage
- `backend/app.py` — HTTP contract

Invariants & edge cases

- Prefer reading code + status docs over README marketing tables when they disagree; mark drift with issues.
- Trademark disclaimer lives on the product index once — not repeated here in full.

Questions

What is PYNE vs pynescript vs pyne?

- **PYNE** — product documentation brand for the open evaluate stack (grammar → AST → runtime → LSP → Pro API → workers).
- **`hoox-pyne`** — **PyPI distribution** name (live, **0.6.7**). Install with `pip install hoox-pyne`. Do **not** install plain `pyne` / `PyNE` from PyPI without the `hoox-` prefix (unrelated) or the upstream `pynescript` package (elbakramer).
- **`pynescript`** — **import package** (`import pynescript`, including `pynescript.runtime`).
- **`pyne`** / **`pyne-lsp`** — preferred console scripts; **`pynescript`** / **`pynescript-lsp`** remain aliases.

What Python versions are supported?

`requires-python = ">=3.10"`. Classifiers list **3.10–3.13**. CI exercises that matrix; pin to 3.10+ for production.

How do I install just the parser?

```
pip install hoox-pyne
```

No extra required for `parse` / `unparse` / `lint` / basic `literal_eval`.

How do I install the LSP?

```
pip install "hoox-pyne[lsp]"
```

Run **`pyne-lsp`** (alias `pynescript-lsp`) — not a Click subcommand of `pyne`.

Why is `pynescript lsp` missing?

The registered entry point is the separate script **`pynescript-lsp`**. Some docs historically mentioned a `lsp` subcommand; trust `pyproject.toml` `[project.scripts]` and `__main__.py` for ground truth.

Which Pine versions are supported?

v6 is the recommended surface (declare `//@version=6`); v5 remains fully supported. See missing features and pine v6 surface.

Does PYNE support the `once` keyword?

Yes, as of **hoox-pyne 0.4.4** (TradingView® August 2026). `once [cond]` runs its block the first time the condition is true on a closed bar. It is **not** an expression — you cannot assign its result. See Expressions & statements.

Is this affiliated with TradingView?

No. Independent open-source toolchain. Pine Script™ and TradingView® are trademarks of TradingView, Inc.

Will my TradingView strategy produce identical PnL?

Not guaranteed. Broker emulator rules, fill models, request.* data, and some builtins differ. Use PYNE for offline analysis, CI, and self-hosted evaluate; validate critical strategies numerically (numerical validation).

Can I format Pine like Black?

`parse-and-unparse` / LSP format normalize via the AST unparser. There is no rich style config comparable to Black's profile system.

Does the CLI evaluate strategies?

`pyne run script.pine --bars 50` compiles and executes on **synthetic** OHLCV. It is **not** `Runtime.run` (no real bars, no `mode` / `libraries` / `timeout_seconds` / `inputs`). For real OHLCV use:

- `from pynescript.runtime import Runtime` — library default `mode` is **interpret**
- `POST /run` — schema default `mode` is **auto**
- `examples/execute_script.py` as a didactic sample (not the production host)

See modes.

How do I lint in CI?

```
pynescript lint --fail-on errors path/to/script.pine
```

Loop files yourself; the CLI is single-file / stdin.

What lint rules exist?

Syntax (`E001`), version (`W001` – `W002`), deprecated patterns (`W101` –), style/naming (`C001` – `C004`). See `src/pynescript/ast/linter.py` and the library API guide.

How do I get market data?

```
pynescript data AAPL --provider yahoo
pip install "hoox-pyne[data]"  # for ccxt
```

Or providers in Python via `pynescript.util.data.get_provider` .

What is the Pro API free tier?

`POST /run` and `POST /run/batch` are exposed as free evaluate endpoints (still your CPU if self-hosted). Default body `mode` is **`auto`** (prefer warm compile; fall back to interpret). Bar/script/rate/concurrency caps are **off unless** you set `FREE_TIER_LIMITS=1` . Responses include `alerts[]` when scripts call `alert()` / true `alertcondition()` (interpret path). Optional L2 webhooks: body `webhook_url` or env `ALERT_WEBHOOK_URL` . Preview/backtest routes are Pro-tier with API keys.

How do interpret and compile compare?

Use the dual-host harness (repo root):

```
python scripts/compare_interp_compile.py --bars 1000 --limit 50
```

See Interpret ↔ compile parity. Pro API `POST /run` defaults to `mode=auto` . The library defaults to `interpret` . Force `interpret` when you need alerts, `input.*` overrides, or full `request.*` .

What file extension should I use?

Prefer **`.pyne`** for HOOX / PYNE stack sources. `.pine` / `.pinev5` / `.pinev6` remain supported by the VS Code extension and parsers.

How complete is real-world script runtime?

PYNE aims for practical fidelity on first-party fixtures and unit tests. It does **not** claim 100% TradingView® platform parity (chart host, proprietary data, every edge-case builtin). See Compatibility. Third-party script corpora are not shipped in this repository.

How do I self-host the API?

Install backend requirements, set `API_KEY_STORE` / `ALLOWED_ORIGINS` / optional `ADMIN_TOKEN` , run `python -m backend.app` or `make run` . See configuration and Pro API usage.

What is AXIS?

Optional charting PWA that can call the evaluate contract and render series. Docs: AXIS. PYNE evaluates without AXIS.

What is HOOX?

Optional edge trading mesh (trade-worker, etc.). Docs: HOOX. Strategy events from PYNE can feed HOOX; not required for parsing/eval.

Alerts: `alert()` / `alertcondition()` firings export on Pro API and pyne-worker `/run`. Optional HTTP webhooks via `webhook_url` or `ALERT_WEBHOOK_URL` (last-bar by default). See Alerts.

What is PyneTS vs pyne-worker vs pyne-agent-worker?

- **PyneTS** (`@hoox-sh/pynets`) — TypeScript / Bun **library + CLI**. `parse` / `unparse` / `Runtime.run`. Not a Worker. Python is the oracle.
- **pyne-worker** — Python Cloudflare Worker. Production `POST /run`. Vendors `pynescript.runtime`.
- **pyne-agent-worker** — NL authoring (`POST /v1/chat`). Standalone chat; optional generate → `/run` validate.

The name **pine-worker** is a leftover TypeScript experiment (hoox-sh/pine-worker). It is not colocated here and is not a product docs target.

See the ecosystem map and modes.

What is the default `mode`?

It depends on the surface — this is the usual footgun:

Surface	Default
`pynescript.runtime.Runtime.run(..., mode=None)`	`PYNE_RUNTIME_MODE` else **interpret**
`POST /run` (body omits `mode`)	**auto**
`pyne run`	**compile-only** (not `Runtime.run`, no `--mode`)

See modes.

Is there `Runtime.evaluate`?

No. The host API is **`Runtime.run`**. Python CLI `pyne run` is compile-only smoke and is not `Runtime.run`. PyneTS `pynets run` **does** call `Runtime.run`.

What license is pynescript?

AGPL-3.0-or-later per `pyproject.toml`. Network use of modified versions requires offering corresponding source (AGPL §13). Proprietary embedding usually needs a commercial license; consult your counsel.

Can I use this commercially?

AGPL permits commercial use under its terms, but distribution or network service of modified versions requires source availability. Hosted Pro API terms (if any) are separate from the library license.

Why is parse slow on huge files?

ANTLR parse + AST build scale with file size and nesting. Deep expression nests also stress recursion (limit temporarily raised to ≥5000). Split libraries when possible.

Round-trip changed my spacing — is that a bug?

Usually no. Unparser emits a normalized style. File a bug if **semantics** change (identifiers, call structure, annotations lost).

Where is the builtin list for autocomplete?

Generated metadata used by the LSP (`scripts/generate_builtin_metadata.py`). Do not hand-edit encrypted release blobs.

How do I report a grammar bug?

Minimal `.pine` repro + expected TV behavior + PYNE version. Prefer failing pytest if contributing.

Is Jupyter supported?

Yes — `from pynescript.ext.jupyter import load_ipython_extension`. The `%%pinescript` cell magic lints, parses, and unparses; it is **not** a bar-loop Runtime. You can also call `parse` / `literal_eval` / `Runtime.run` directly in notebooks.

Can I transform scripts programmatically?

Yes — `NodeVisitor` / `NodeTransformer` on the AST, then `unparse`. See library API.

Worked examples

“Is my install healthy?”

```
python -c "from pynescript.ast import parse, unparse; print(unparse(parse('//@version=6\nindicator(
pynescript lint - <<< $'//@version=6\nindicator("t")\nplot(close)\n'
```

“Does HTTP evaluate work?”

```
curl -s http://127.0.0.1:5002/ | python -m json.tool
```

“Is LSP importable?”

```
python -c "import pygls; from pynescript.langserver.server import PynescriptLanguageServer"
```

Failure modes

If an FAQ answer appears wrong against your checkout:

1. Check package version (`pynescript --version`).
2. Read the cited source file.
3. Prefer `__main__.py` / `pyproject.toml` over secondary docs.
4. Open an issue or PR against the MDX page.

See also

- End User hub
- Installation
- Troubleshooting
- Glossary
- Compatibility
- CLI commands